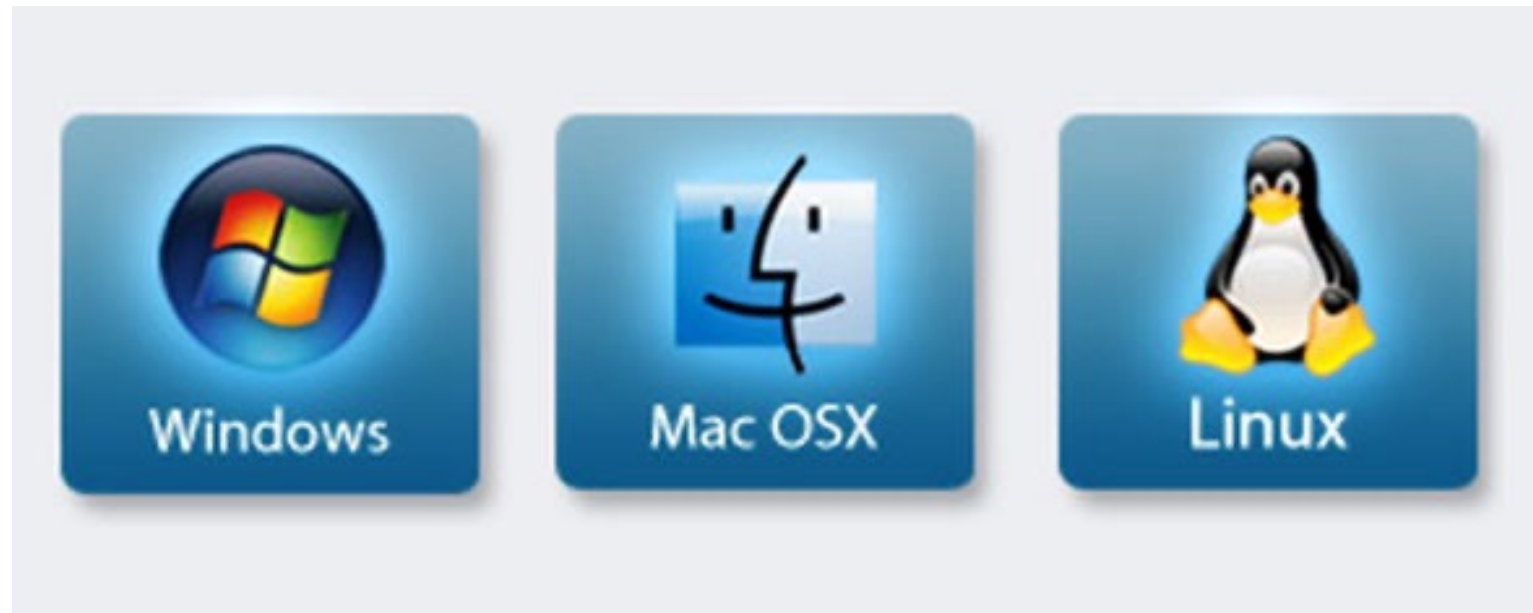




HE1025 Operativsystem

F1: Virtualisering av Processorn, mekanismer.

Operativsystem?



Windows NT Kernel

XNU

BASH

Inbyggda system



Varför?

- Det finns (nästan) inget inom området datateknik som inte utnyttjats till det yttersta för att skapa dagens högpresterande operativsystem...
- ...den som har förstått principerna för hur ett operativsystem är uppbyggt har med andra ord tagit till sig grunderna för (nästan) hela kunskapsområdet...
- ...att skapa program i dag är att tillämpa valda delar av insikt från operativsystemsutvecklingen inom nya områden!

Resource allocation, Reliability/Privacy/Security & Fairness/Efficiency



Kursen...

F1: V#1 Processorn.

Ö1: V#2 c Pekare...

F2: V#3 Schemaläggning

L1: V#4 FreeRTOS

F3: V#5 Segmentation

Ö2: V#6 Eget arbete

F4: V#7 Paging

L2: V#8 Paging

F5: C#1 Locks & CV

Ö3: C#2 LINUX

F6: C#3 Semaphores

L3: C#4 Barberare

F7: P#1 Storage

Ö4: P#2 File/DMA

F8: P#3 Communication

L4: P#4 Client/Server

Kap (1) 2..4 & 6

Kap 14 & c-boken

Kap 7..9,(10-11), RTOS

Lab #1 Handledning

Kap 12..13, 15..17

-

Kap 18-22, (23-24)

Lab #2 Handledning

Kap (25) 26..27 & 28..29.1

Kap 5 (39)

Kap 30..33, (34)

Lab #3 Handledning

Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)

-

Kap (47, 39) & 48, (49..51)

Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Redovisning Lab 1,3&4, närvaro L2.



Bare metal... (...nej, vi kommer inte att tala om hårdrock!)

Applikationsspecifik kod

Återanvändbar kod, bibliotek!

...den utvecklade koden innehåller "allt" och körs direkt mot den "nakna hårdvaran" (=bare metal)!

Används i dag för:

- Inbyggda system (litet)
- Serverlösningar (snabbt)



HE1028
(HI1024)

--- API ---

HW

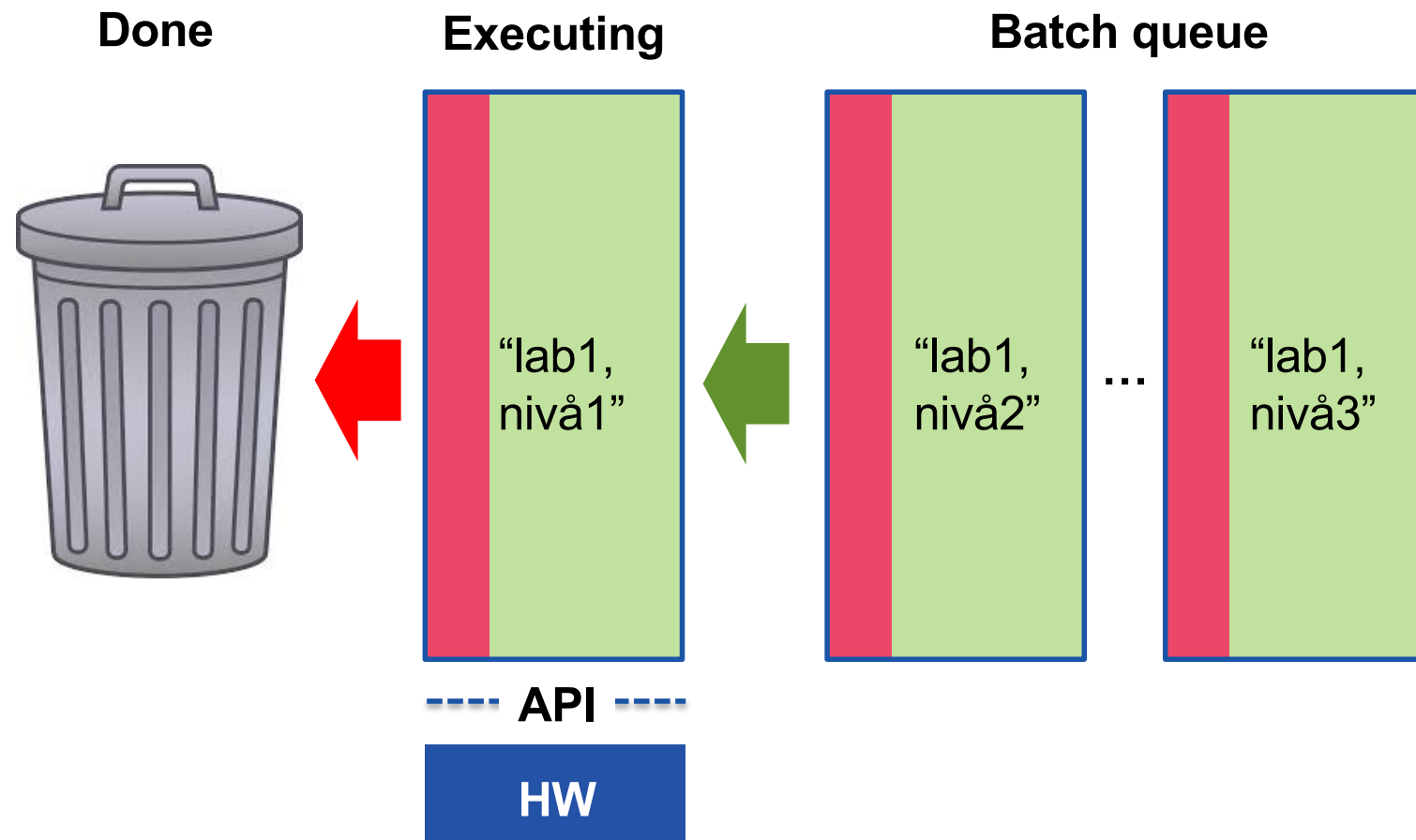
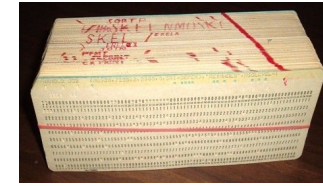
Tillverkarspecifikt API (ISA):

- Processorns maskinkod
- Kortets hårdvara

(Instruction Set Architecture)

Helt ok, men bara ett program kan köra åt gången, och vill vi köra på en annan hårdvara så måste nästan allt skrivas om!

Batch Job



I datorns barndom var detta en mycket vanlig företeelse...
...men väldigt kraftfull, och principen finns kvar än i dag!



NASA 1957:

I förgrunden IBM 711 kortläsare, bakom den en IBM 704 dator!



Comand Line Interface (CLI)

Redan de första datorerna hade någon form av CLI som möjliggjorde rudimentär administration av datorn...

...typiskt fanns kommandon för att administrera hur datorn exekverade program (vart de fanns, in-/utdata filer)...

...för att inte behöva sitta framför datorn så kunde CLI kommandon skrivas efter varandra i en fil som då blev ett skript (manus) för vad datorn skulle göra under natten...

...en batch av jobb som var klar på morgonen!

Ett batch job för en CDC dator (-70)
(Skriva ut data från magnetband)

```
VSCSTP.  
USER,VSCBAP,VSCSCI.  
GET,STARTRK/UN=EDUC.  
REWIND,STARTRK.  
1 GET,PNCHPM/UN=VSC1,PW=CARD.  
2 REWIND,PNCHPM.  
3 COPYBR,PNCHPM,PNCH.  
4 COPYSBF,STARTRK,PNCH.  
5 COPYBF,PNCHPM,PNCH.  
6 REWIND,PNCH.  
7 COPY,PNCH,OUTPUT.  
8 "  
9
```




Windows CMD/.bat

Windows föregångare hade en kommandotolk...
...och möjligheten att starta, till och med schemalägga job!

Kommandotolken finns kvar än i dag...
...du hittar den under namnet terminal...
...och kommandon kan samlas i en .bat-fil...
...för att automatiskt utföras vid ett senare tillfälle!



Linux Shell/Shell script

LS(1) BSD General Commands Manual LS(1)

NAME

`ls -- list directory contents`

SYNOPSIS

`ls [-ABCFGHLOPRSTUW@abcdefghijklmnopqrstuvwxyz1] [file ...]`

DESCRIPTION

For each operand that names a file of a type other than directory, `ls` displays its name as well as any requested, associated information. For each operand that names a file of type directory, `ls` displays the names of files contained within that directory, as well as any requested, associated information.

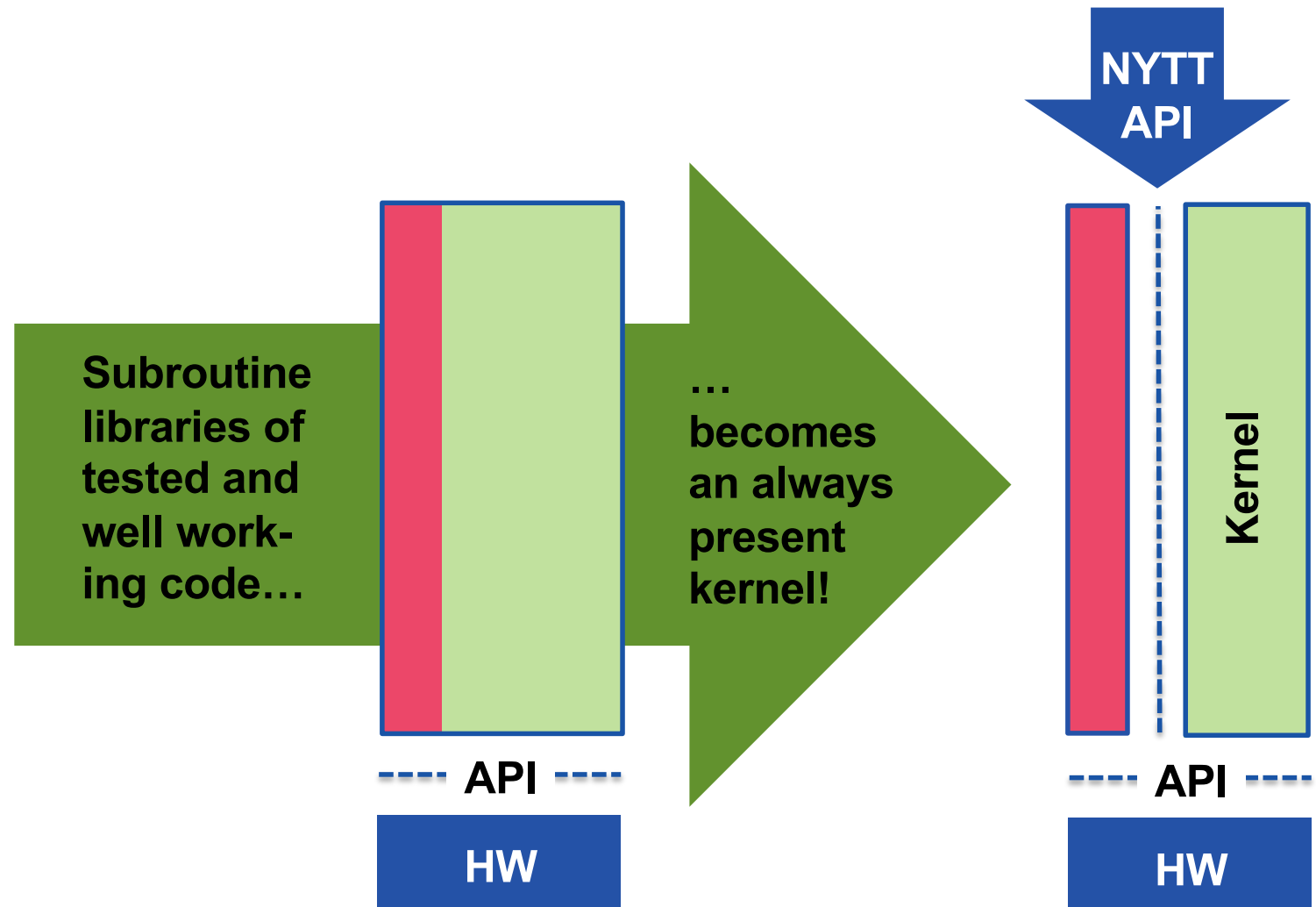
If no operands are given, the contents of the current directory are displayed. If more than one operand is given, non-directory operands are displayed first; directory and non-directory operands are sorted separately and in lexicographical order.

The following options are available:

- `-@` Display extended attribute keys and sizes in long (`-l`) output.
- `-1` (The numeric digit ``one''.) Force output to be one entry per line. This is the default when output is not to a terminal.
- `-A` List all entries except for `_` and `..`. Always set for the super-user.

Sparas i en fil som märks som "körbar"

Subrutinbiblioteket blir något mer...



Intressant, "allt" komplicerat göms bort bakom ett nytt API!
(Benämns Application Binary Interface på den lägsta nivån)

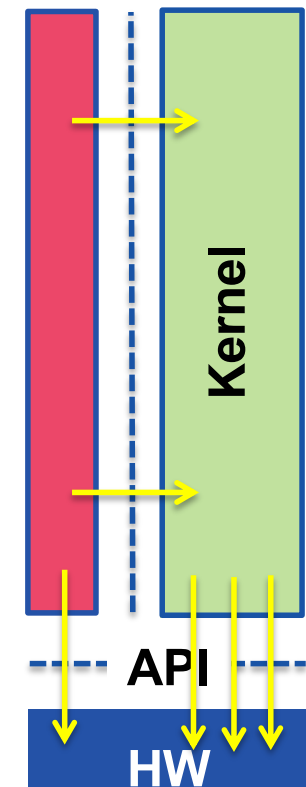


Subrutinbiblioteket blir något mer...

Den applikationsspecifika koden exekverar direkt på processorn...

...och kan anropa kärnrutiner som kör direkt på processorn, men applikationskod kan inte (lätt) röra kärnrutinernas datastrukturer!

Direct Execution!





Subrutinbiblioteket blir något mer...

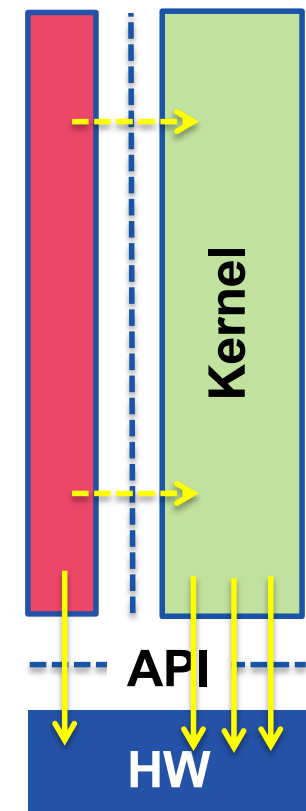
Den applikationsspecifika koden exekverar direkt på processorn...

...och kan anropa kärnrutiner som kör direkt på processorn, men applikationskod kan inte (lätt) röra kärnrutinernas datastrukturer!

Det finns dock uppenbart instruktioner som applikationskoden inte borde få utföra...

...det kan vi inte hantera utan hårdvarustöd!

Limitid Direct Execution!



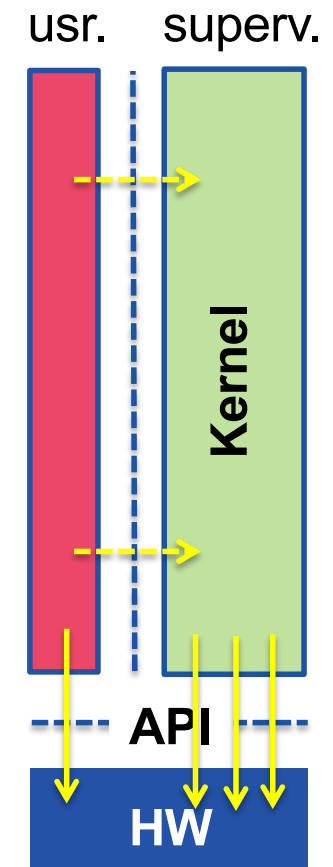
Subrutinbiblioteket blir något mer...

Limitid Direct Execution!

1

Består av **två delar** där den första innebär att dagens moderna processorer kan arbeta i (minst) två moder, där den minst privilegierade moden (bara "ofarliga" maskininstruktioner är tillåtna) ofta kallas user mode och den mest tillåtande moden ofta benämns supervisor/kernel/machine mode...

...den andra mekanismen beskriv i slutet av föreläsningen.



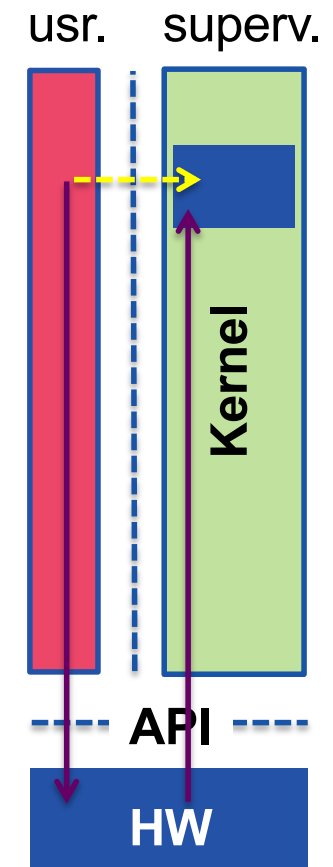


Subrutinbiblioteket blir något mer...

Att anropa kärnan blir nu lite mer komplicerat då processorn borde byta till kernel mode när det sker...

1

...det vanliga funktionsanropet ersätts av ett så kallat System Call, som i själva verket är ett avbrott (interrupt/trap) som via en boot-initierad hw-tabell anropar rätt kärnrutin samtidigt som processorn byter till kernel-mod. När kärnrutinen är klar utför den en speciell privilegierad "return from trap" instruktion som återställer processorn till user mode och instruktionen efter anropet!



Det här är inte hela sanningen, och även om vi inte tar resten så är det uppenbart att processen tar lite tid (i dag ca 0.1ms) men det är priset för att kunna skapa nödvändig säkerhet (isolation)!



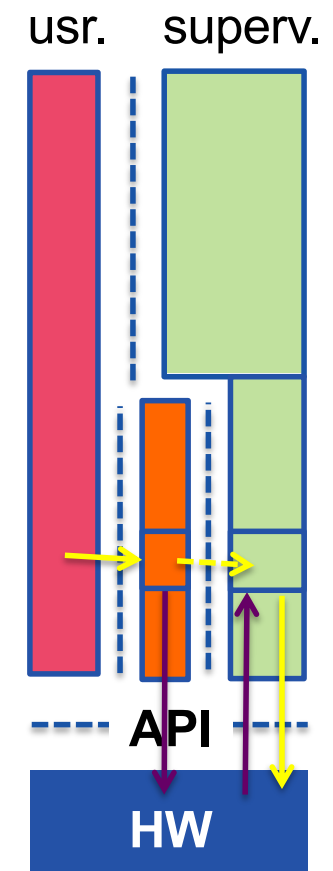
Subrutinbiblioteket blir något mer...

Alla kärnfunktioner kräver inte kernel mode så delar av kärnan (vissa api:n) finns till vänster om kärn api:et och kan då anropas snabbt som vanliga funktioner...

1

...men då utan det skydd som ett kärnanrop ger!

När vi i c anropar <xxxx.h> bibliotek så gör vi det i user mode. Vissa av biblioteken utför, när det så behövs, system calls, som för en stund flyttar exekveringen till kärnan i kernel mode.



(Vi behåller den förenklade bilden för stunden)

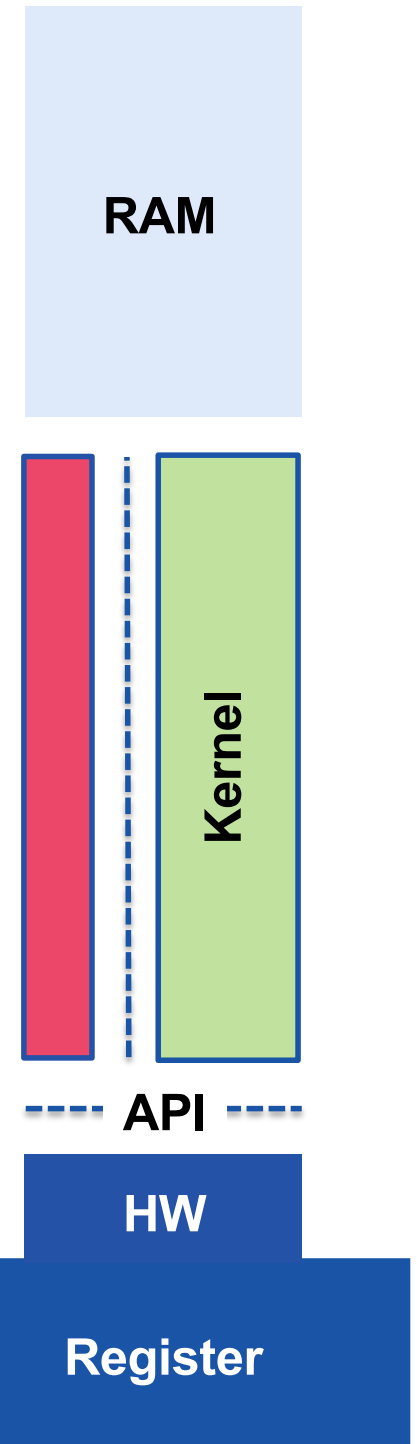


Subrutinbiblioteket blir något mer...

När applikationsprogrammet exekverar så innehåller uppenbart processorns register information som är kopplat till programmet...

...applikationsprogrammet behöver säkert också eget ram-minne för att (mellan-)lagra resultat!

M.a.o så måste processen tilldelas minne, och processorregistren måste initieras.





Bra, men det finns fler krux...

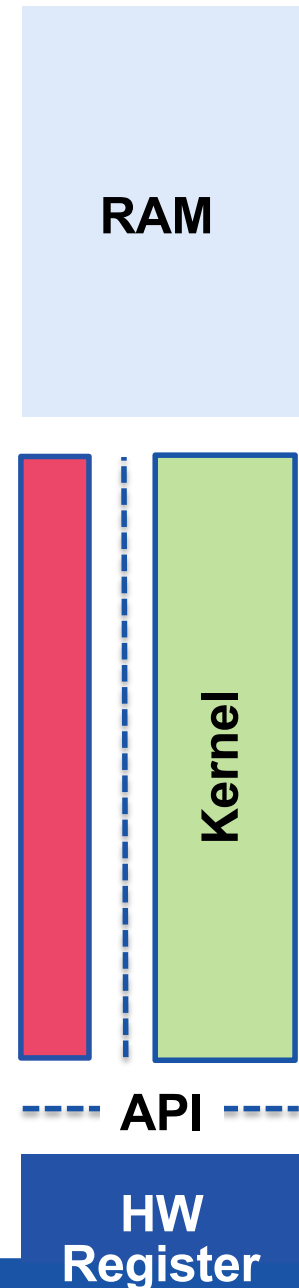
Vi har nu en bra uppdelning där vi "enbart" behöver utveckla applikationsspecifik kod, som på ett säkert sätt anropar robusta kärnrutiner via system calls...

...men main loop anropar kanske 20 funktioner som i tur och ordning, helt kort, kan utföra sina funktioner. Skulle någon funktion krascha slutar ofta hela programmet att fungera...

...det är kanske också så att en del av funktionerna är tidskritiska och måste anropas exakt varje ms...

...arbetar någon rutin, via kärnanrop, mot en hård-disk kan den behöva vänta motsvarande 100.000-tals instruktioner innan den får fortsätta...

...Hum, om applikationen i stället kunde utvecklas som ett antal program som skenbart körde parallellt skulle mycket vara vunnet!





#1: VIRTUALIZATION

Krux...

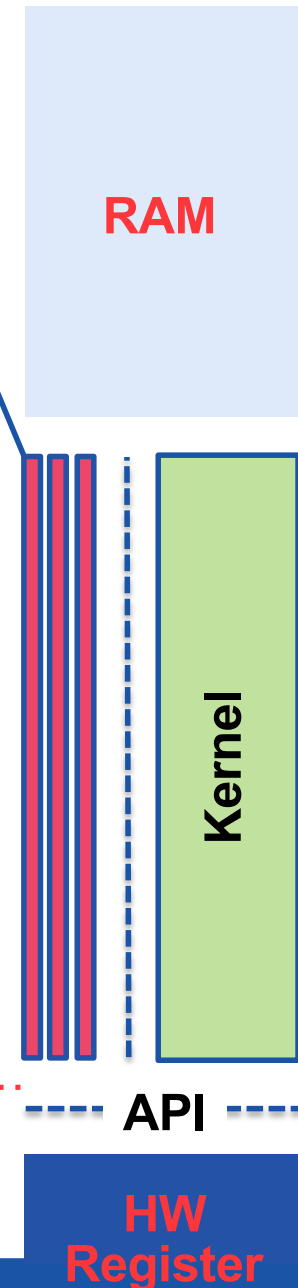
Ursprungsprogrammet nu uppdelat i ett antal små samverkande program!

Skulle vi i stället kunna skapa en mängd "program"...

- ...som till synes exekverade parallellt...
- ...som kunde kommunicera med varande vid behov...
- ...så skulle inte allt sluta fungera om ett "program" kraschade!
- ..."program" som måste köras vid speciella tidpunkter kan schemaläggas så att det sannolikt sker!
- ...om ett "program" behövde vänta på data från t.ex. en hård disk, så kan ett annat "program" köra under tiden!

Det finns uppenbart bara en (ett fåtal) processor(er)...

- ...kärnan måste ge sken av att det finns många cpu:er...
- ...kärnan måste virtualisera processorn!
- ...minnet måste också virtualiseras!



Att flera program kan köra (tillsynes) samtidigt är en mekanism. Vilket program som körs vid någon tidpunkt beror på en policy!



#2: CONCURRENCY

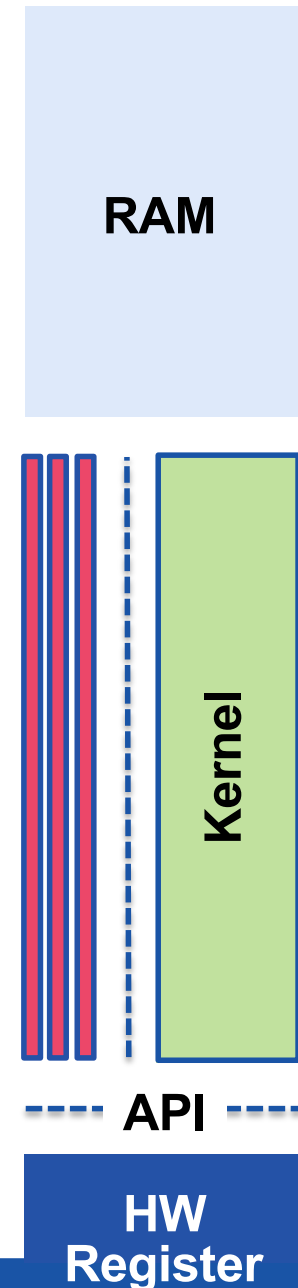
Krux...

När flera program skenbart exekverar samtidigt kan det uppstå många märkliga, svårfunna & fatala fel...

Ta 2 program som bägge behöver 2 olika resurser för att utföra nästa uppgift. Det ena programmet har den ena resursen och väntar på att kunna ta den andra resursen, men den ägs redan av det andra programmet som väntar på den första resursen...

Ta 2 program som använder samma variabel, säg int count. Bägge programmen utför `count=count+1`; vid olika tillfällen. Vi kommer i kursen att se att den beräkningen nästan aldrig blir rätt...

Vad händer om flera program vill köra samma kod...



Ett operativsystem kan inte parera för alla dessa problem så den utvecklade koden måste ta hänsyn till ett antal av problemen!



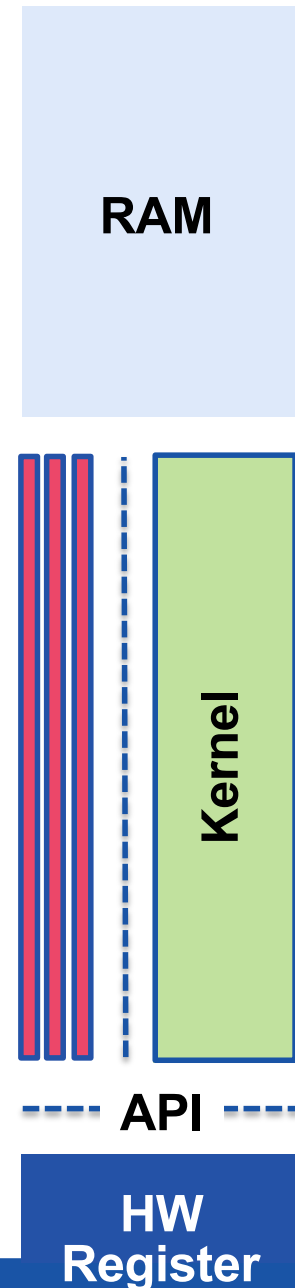
#3: PERSISTENCE

Krux...

Den del av ett operativsystem som arbetar mot anslutna enheter som kan spara information permanent kallas ofta filsystem...

...det tar tid att skriva, eller läsa, från en fil så moderna operativsystem använder flera mekanismer för att det ska gå så snabbt som möjligt...

...tyvärr så innebär det också att ännu fler mekanismer måste vidtas för att säkerställa att informationen i filsystemet är konsistent om något skulle hända mitt i en operation!



Men kan vår batch monitor uppgraderas med lösningar för virtualization, concurrency & persistence så inträffar 2 saker...

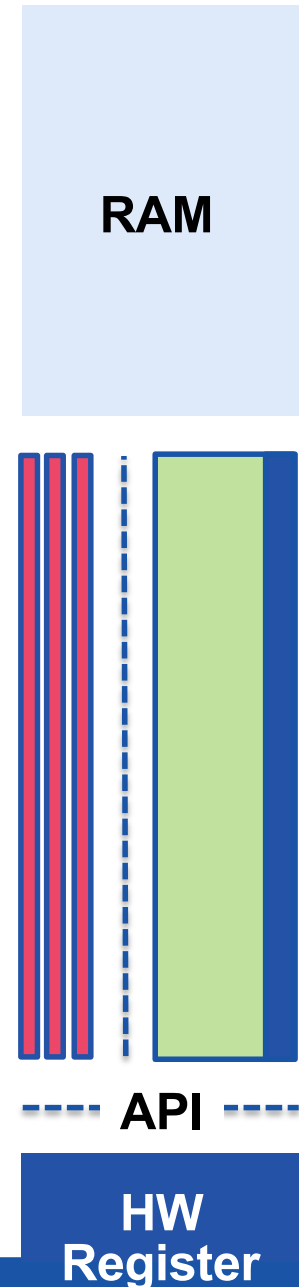


1: Wow, portabelt!

Huvuddelen av kärnan är säkert kod som är oberoende av den underliggande hårdvaran...

...vanligtvis samlas den kod som är HW-specifik i ett fåtal filer...

...om kärnan är skriven i t.ex. c så krävs enbart en c-kompilator och att de beroende filerna uppdateras, för att kärnan ska fungera på den nya hårdvaran, och indirekt att alla existerande program därmed också fungerar!



För att flytta LINUX, 20+ miljoner rader kod, till en ny plattform...
...behövs vanligtvis mindre än 4000 rader uppdateras!

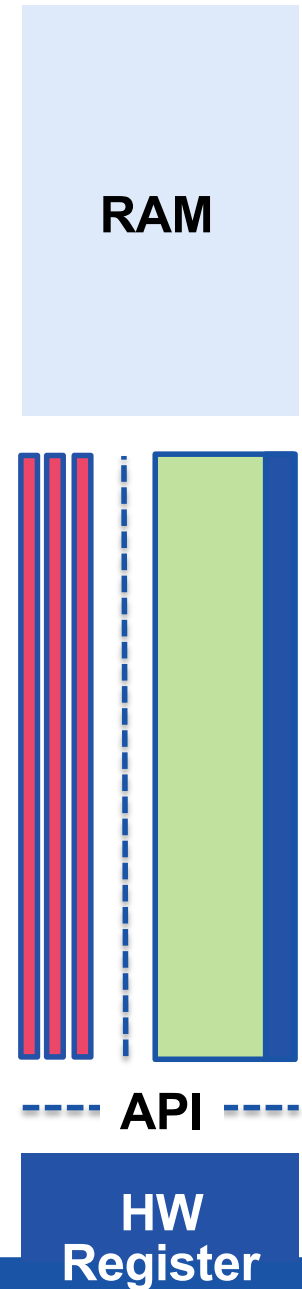


2: Wow, flera användare!!!

Fram till nu har vi använt möjligheten att köra flera program (till synes) parallellt för att på ett bättre sätt implementera vårt ursprungsprograms main loop...

...men det tog inte länge innan branschen insåg att kunde kärnan köra flera program "parallellt" så skulle datorn (till synes) kunna betjäna flera användare samtidigt!

Under -60-talet skedde denna övergång från Batch computing till så kallade Time Sharing system. Kärnan blev nu ett Operativsystem!



I dag benämns möjligheten multiprogramming.





Design principer...

Att designa ett OS är inte lätt...

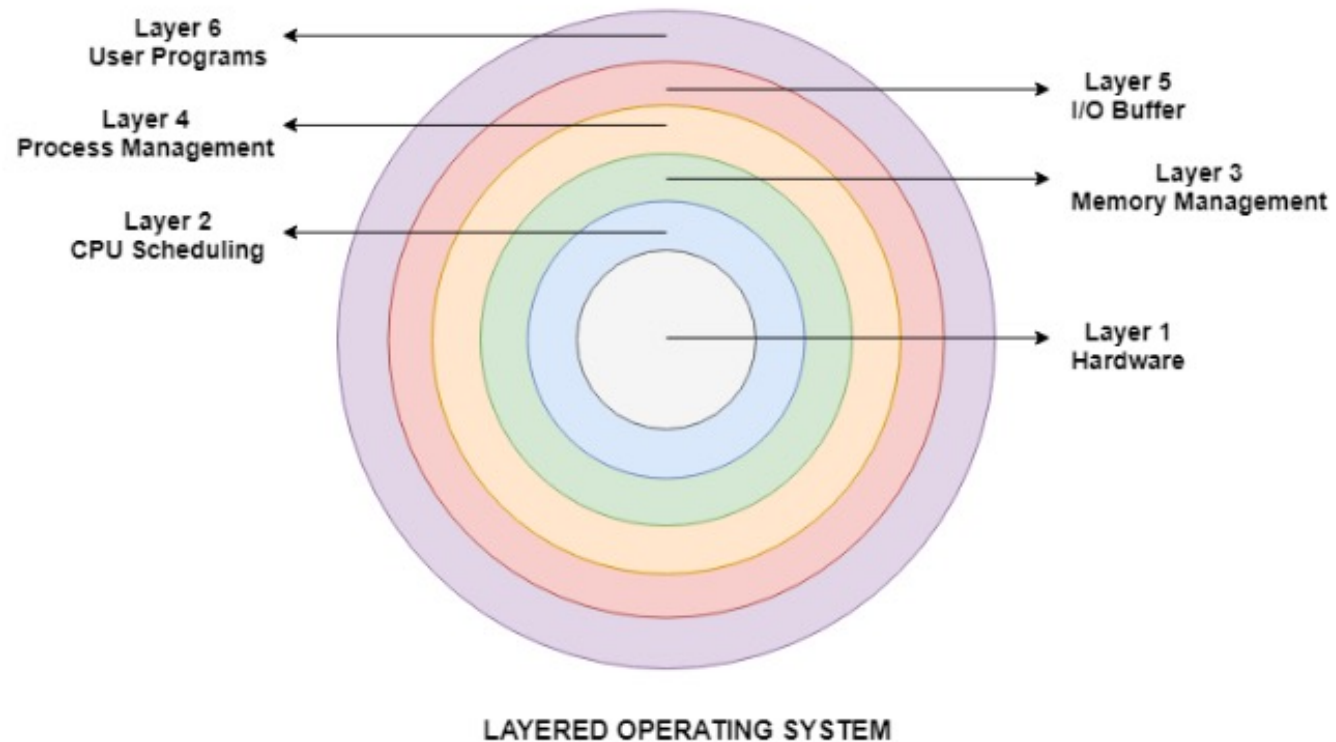
...bärande ide är abstraktion där ett väl valt API, med några 100 funktionsanrop, erbjuder hårdvarans resurser virtualiserade – stöd för att hantera parallellism och stöd för hantering av filer (virtualization, concurrency & persistenc).

Det här ska helst ske med liten overhead, för maximal prestanda!

Andra bärande principer är: Isolation, tillförlitlighet, säkerhet, energi effektivitet och mobilitet.

Den som utvecklar program för os:et vill säkert också att det om möjligt är standardiserat, eller i alla fall väldigt spritt!

Lökmodellen...



Abstraktion är en vän, men i detta fall blir det lite svårt att rita...
...då en tjänst ofta består av både mekanismer och policys!

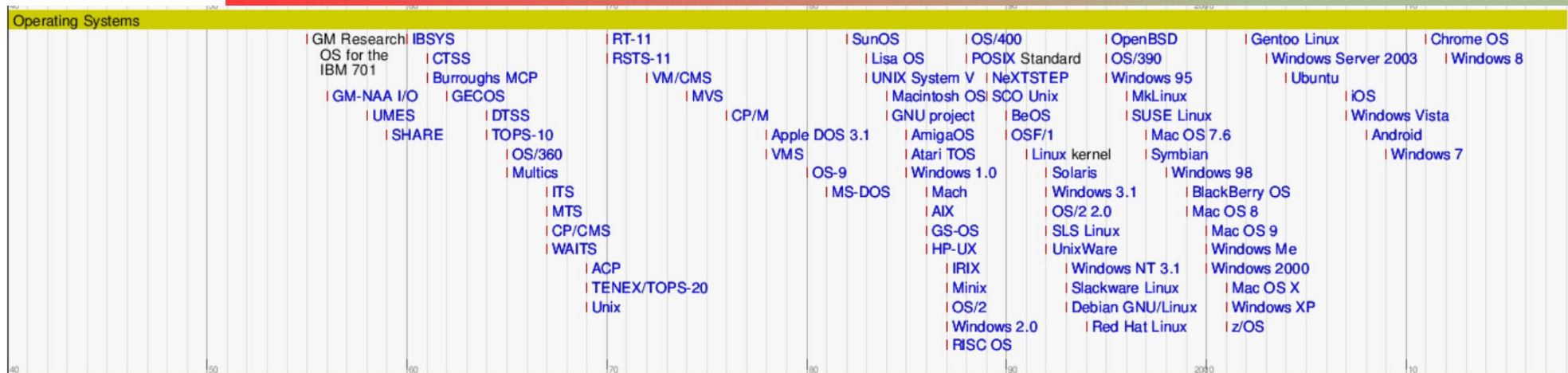


OS timeline

Alltid leverantörsspecifika

WINDOWS

"UNIX" VARIANTER

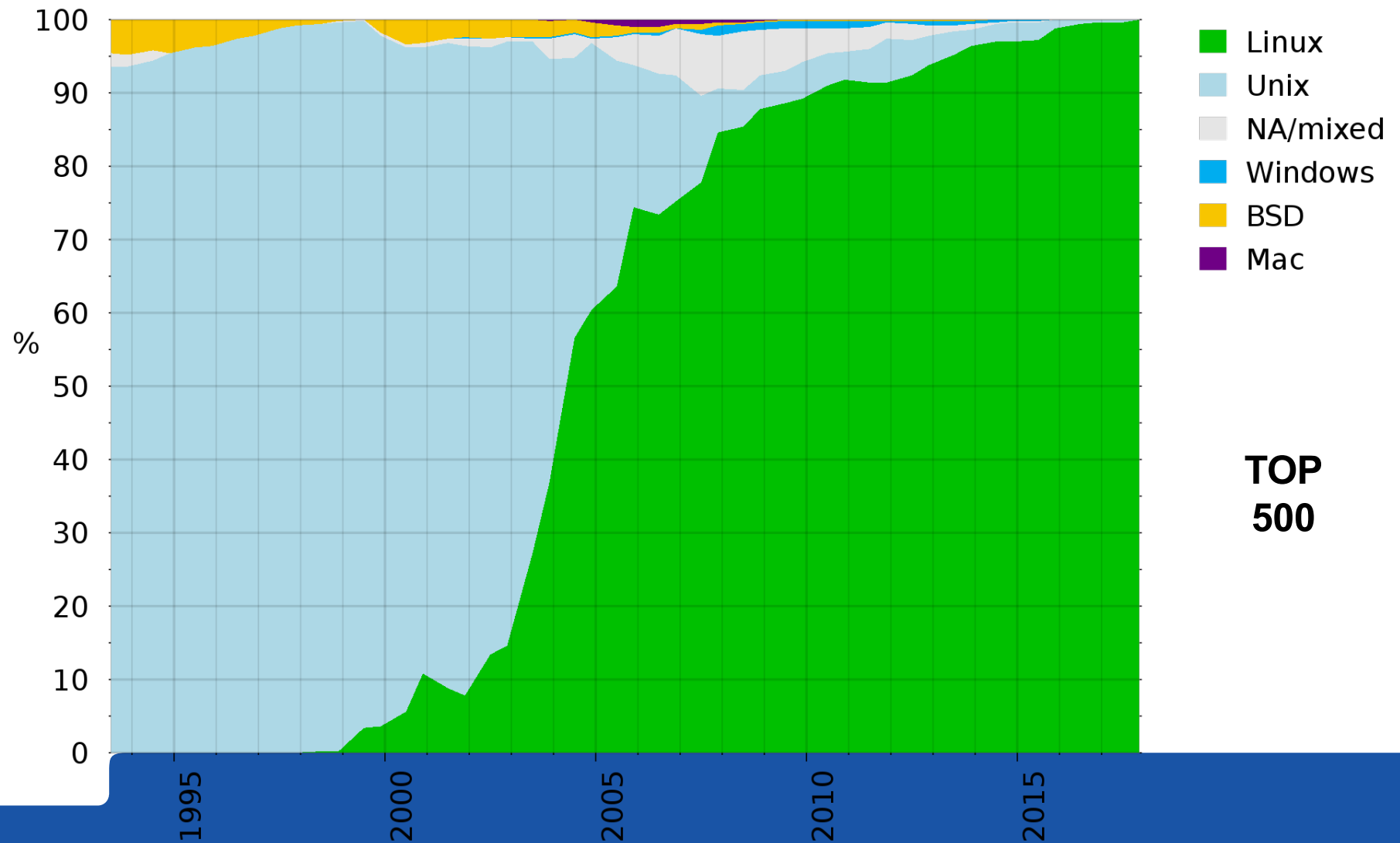


RTOS

(FreeRTOS)



OS timeline



**TOP
500**



Virtualization...

HI1024

Grundkurs prg.

När vi skriver ett program i assembler måste vi ha absolut kontroll över hur minnet & processorn används!

Utvecklar vi ett program i c, eller något annat högnivåspråk är det inte lika uppenbart hur kompilatorn väljer att använda tillgängligt minne och processor...

...ska vi förstå vad ett operativsystem måste kunna hantera, för att ge illusionen av att systemet kan köra flera program parallellt, så måste vi förstå mer om den kod som kompilatorn skapar för ett program...

Detta gäller speciellt för minnesbegränsade inbyggda system!



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

HI1024
THE MISSING
CHAPTER!

```
4  _fak:                                     ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)          ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)          ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax          ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                     ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)          ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

DATA (Gl.var)

TEXT (CODE)

Low mem addr



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                     ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)          ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)          ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax          ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                     ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)          ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

STACK

DATA (Gl.var)

TEXT (CODE)

Low mem addr

Stack Frame



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                     ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)          ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)          ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax          ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                     ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)          ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

n = 3
return address

DATA (Gl.var)

TEXT (CODE)

Low mem addr

Stack Frame



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                     ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     movl    -4(%rbp), %eax
25     movl    %eax, -8(%rbp)          ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)          ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax          ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                     ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)          ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

n = 3
return address

n = 2
return address

DATA (Gl.var)

TEXT (CODE)

Low mem addr

Stack Frame



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                     ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)                ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)                ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax                ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl _main
39     .align  4, 0x90
40     _main:                                ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)                ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

n = 3
return address

n = 2
return address

n = 1
return address

DATA (Gl.var)

TEXT (CODE)

Low mem addr

Stack Frame



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)      ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)      ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax      ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)      ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

n = 3
return address

n = 2
return address

n = 1
return address

n = 0
return address



DATA (Gl.var)

TEXT (CODE)

Low mem addr



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)          ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)          ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax          ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)          ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

n = 3
return address

n = 2
return address

n = 1
return address



DATA (Gl.var)

TEXT (CODE)

Low mem addr



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                     ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)          ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)          ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax          ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl _main
39     .align 4, 0x90
40     _main:                                ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)          ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

n = 3
return address

n = 2
return address



DATA (Gl.var)

TEXT (CODE)

Low mem addr



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)      ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)      ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax      ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)      ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2

n = 3
return address

6

DATA (Gl.var)

TEXT (CODE)

Low mem addr



Begrepp

```
#include <stdio.h>

int fak(int n){
    return n ? fak(n-1)*n : 1;
}

int main(int argc, char **argv)
{
    int nb1 = 3;
    int nb2;

    nb2 = fak( nb1 );

    printf("%d",nb2);
    return 0;
}
```

```
4  _fak:                                     ## @fak
5      .cfi_startproc
6      ## BB#0:
7      pushq   %rbp
8      Ltmp2:
9      .cfi_def_cfa_offset 16
10     Ltmp3:
11     .cfi_offset %rbp, -16
12     movq    %rsp, %rbp
13     Ltmp4:
14     .cfi_def_cfa_register %rbp
15     subq    $16, %rsp
16     movl    %edi, -4(%rbp)
17     cmpl    $0, -4(%rbp)
18     je      LBB0_2
19     ## BB#1:
20     movl    -4(%rbp), %eax
21     subl    $1, %eax
22     movl    %eax, %edi
23     callq   _fak
24     imull   -4(%rbp), %eax
25     movl    %eax, -8(%rbp)          ## 4-byte Spill
26     jmp     LBB0_3
27     LBB0_2:
28     movl    $1, %eax
29     movl    %eax, -8(%rbp)          ## 4-byte Spill
30     jmp     LBB0_3
31     LBB0_3:
32     movl    -8(%rbp), %eax          ## 4-byte Reload
33     addq    $16, %rsp
34     popq    %rbp
35     retq
36     .cfi_endproc
37
38     .globl  _main
39     .align  4, 0x90
40     _main:                                     ## @main
41     .cfi_startproc
42     ## BB#0:
43     pushq   %rbp
44     Ltmp7:
45     .cfi_def_cfa_offset 16
46     Ltmp8:
47     .cfi_offset %rbp, -16
48     movq    %rsp, %rbp
49     Ltmp9:
50     .cfi_def_cfa_register %rbp
51     subq    $32, %rsp
52     movl    $0, -4(%rbp)
53     movl    %edi, -8(%rbp)
54     movq    %rsi, -16(%rbp)
55     movl    $3, -20(%rbp)
56     movl    -20(%rbp), %edi
57     callq   _fak
58     leaq    L_.str(%rip), %rdi
59     movl    %eax, -24(%rbp)
60     movl    -24(%rbp), %esi
61     movb    $0, %al
62     callq   _printf
63     movl    $0, %esi
64     movl    %eax, -28(%rbp)          ## 4-byte Spill
65     movl    %esi, %eax
66     addq    $32, %rsp
67     popq    %rbp
68     retq
69     .cfi_endproc
```

High mem addr

Environment var.
Argc & argv
nb1 = 3
nb2 = 6

DATA (Gl.var)

TEXT (CODE)

Low mem addr

c funktioner är
REENTRANT



Hum...

INT
250ms

Blinkar
0x55-0xAA

```
*** DELAY Package ***** 1.0 * AND *
IDSHORT equ    0x070      ; Delay short counter
IDLONG  equ    0x071      ; Delay long counter
IDVERYL equ    0x072      ; Delay very long counter

ED100MS CLRf    IDSHORT    ; Reset delay counters.
        MOVLW    0xC0
        MOVWF    IDLONG
LDSLOOP DECFSZ   IDSHORT    ; Count down...
        GOTO     LDSLOOP   ; ..until zero!
        DECFSZ   IDLONG     ; Count down again...
        GOTO     LDSLOOP   ; ..until zero!
        RETURN

ED200MS MOVLW    0x02
        MOVWF    IDVERYL
LDLLOOP CALL     ED100MS    ; Wait
        DECFSZ   IDVERYL    ; Count down...
        GOTO     LDLLOOP   ; ..until zero!
        RETURN

*** End of DELAY Package *****

*** MAIN *****
MINIT   BCF      TRISA,RA1    ; COL1 out & on
        BSF      PORTA,RA1
        CLRf     TRISD      ; ROWS out & 01010101
        MOVLW    0x55
        MOVWF    PORTD
        CALL     ETSTART    ; Start timer!

        BSF      INTCON,GIE_GIEH --< Allow INTERRUPT! >--

MLOOP   COMf     PORTD        ; Toggle rows...
        CALL     ED200MS     ; (wait)
        GOTO     MLOOP      ; ...forever!
```



Hum...

INT
250ms

ED100MS är INTE reentrant!

Blinkar
0x55-0xAA

```
*** DELAY Package ***
IDSHORT equ    0x070
IDLONG  equ    0x071
IDVERYL equ    0x072
; Delay very long counter

ED100MS CLRf    IDSHORT    ; Reset delay counters.
        MOVLW    0xC0
        MOVWF    IDLONG
LDSLOOP DECFSZ   IDSHORT    ; Count down...
        GOTO     LDSLOOP    ; .. until zero!
        DECFSZ   IDLONG     ; Count down again...
        GOTO     LDSLOOP    ; .. until zero!
        RETURN

ED200MS MOVLW    0x02
        MOVWF    IDVERYL

*** End of DELAY Package ***

*** MAIN ***
MINIT   BCF      TRISA, RA1    ; COL1 out & on
        BSF      PORTA, RA1
        CLRF     TRISD        ; ROWS out & 01010101
        MOVLW    0x55
        MOVWF    PORTD
        CALL     ETSTART      ; Start timer!

        BSF      INTCON, GIE_GIEH --< Allow INTERRUPT! >--

MLOOP   COMF     PORTD        ; Toggle rows...
        CALL     ED200MS      ; (wait)
        GOTO     MLOOP        ; ...forever!
```

Kommer alltid att
vara 0 efter avbrott!

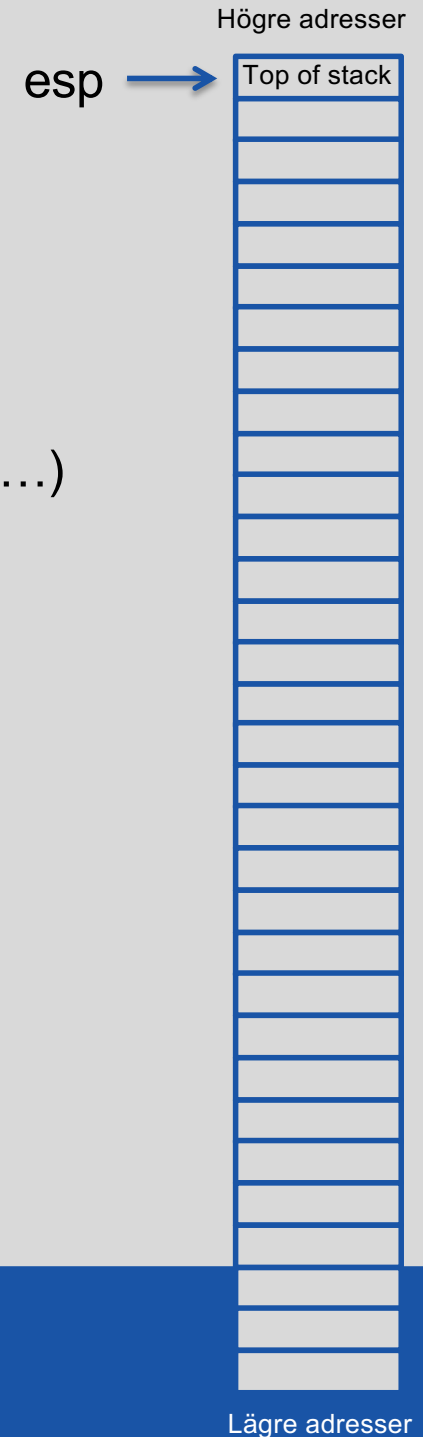


Fördjupning: stack frame

Komp. lägger ut kod vid ett funktionsanrop
som arbetar med 2 register:

Stackpointer: intel esp (ARM R13, RISC-V X2...)

Basepointer: intel ebp





Fördjupning: stack frame

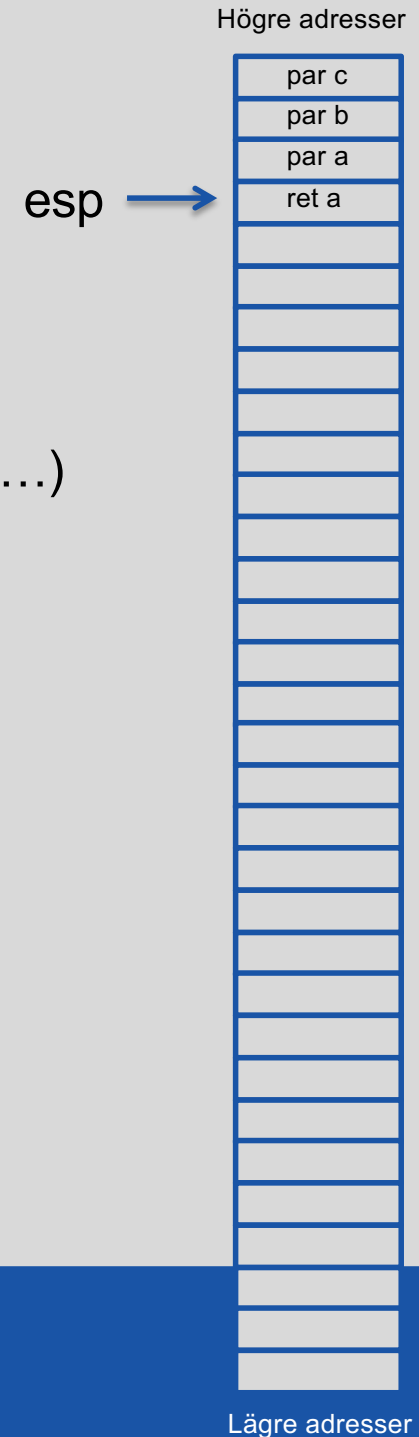
Komp. lägger ut kod vid ett funktionsanrop
som arbetar med 2 register:

Stackpointer: intel esp (ARM R13, RISC-V X2...)

Basepointer: intel ebp

Om en funktion med 3 int parametrar och
3 lokala int variabler anropas så...

...trycks först par. på stacken därefter call...





Fördjupning: stack frame

Komp. lägger ut kod vid ett funktionsanrop som arbetar med 2 register:

Stackpointer: intel esp (ARM R13, RISC-V X2...)

Basepointer: intel ebp

Om en funktion med 3 int parametrar och 3 lokala int variabler anropas så...

...trycks först par. på stacken därefter call...

...därefter sparas nuvarande ebp på stacken...

...ebp tilldelas esp, esp minskas med 3×4 ...

! Lätt att komma åt parametrar/variabler

Lätt att anropa nya funktioner

Då anropande funktions context finns via $*(ebp)$...
...kan en ögonblicksbild lätt skapas vid felsökning!





Fördjupning: stack frame

Komp. lägger ut kod vid ett funktionsanrop som arbetar med 2 register:

Stackpointer: intel esp (ARM R13, RISC-V X2...)

Basepointer: intel ebp

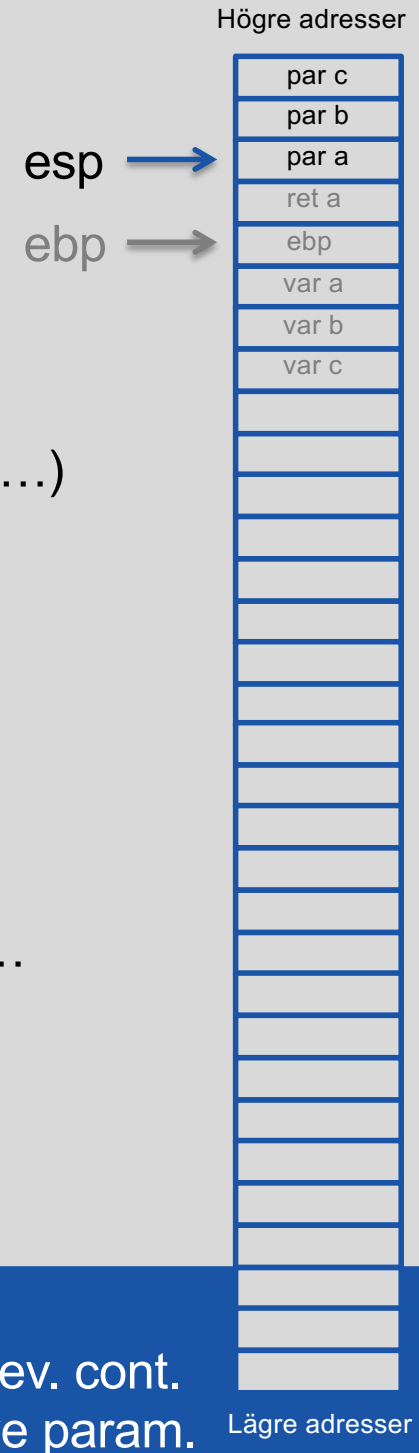
Om en funktion med 3 int parametrar och 3 lokala int variabler anropas så...

...trycks först par. på stacken därefter call...

...därefter sparas nuvarande ebp på stacken...

...ebp tilldelas esp, esp minskas med $3 \cdot 4$...

! Lätt att komma åt parametrar/variabler
Lätt att anropa nya funktioner





Fördjupning: stack frame

Det här är också förklaringen till att c-operatorn `sizeof()` inte ger förväntat resultat när den används på en parameter av array-typ...

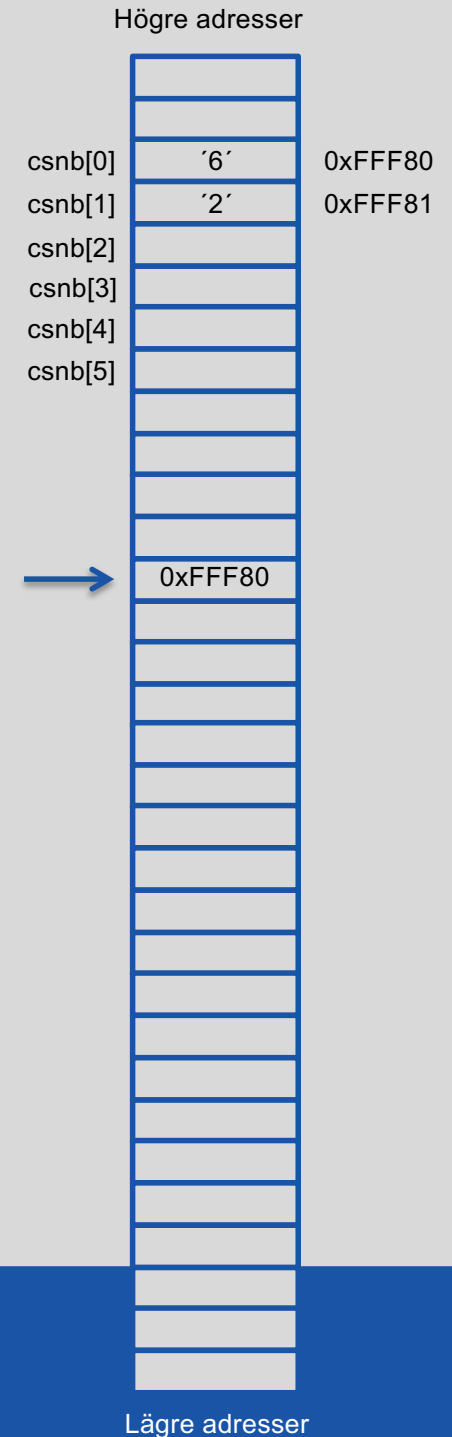
...säg att din kod vill anropa en funktion som `esp` har som sin första parameter en char array:

```
void printCSNb(char aCSNb[]...) {...}
```

...funktionen anropas med `printCSNb(csnb...);`

...hela arrayen trycks då inte på stacken utan det enda som förmedlas är adress till första elementet i arrayen, mao en pekare!

sizeof(aCSNb) returnera vanligtvis 4!





Övning: Ok?

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {char title[20+1]; char category[20+1];} Book;

Book *init(char theTitle[], char theCategory[]) {
    Book theBook;
    strcpy(theBook.title, theTitle);
    strcpy(theBook.category, theCategory);
    return &theBook;
}

int main(int argc, char **argv)
{
    Book *pBook=init("Foundation","Science Fiction");
    printf("Book #1: %s %s\n", pBook->title, pBook->category);
    return 0;
}
```



Övning: Ok?

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {char title[20+1]; char category[20+1];} Book;

Book *init(char theTitle[], char theCategory[]) {
    Book theBook;
    strcpy(theBook.title, theTitle);
    strcpy(theBook.category, theCategory);
    return &theBook;
}

int main(int argc, char **argv)
{
    Book *pBook=init("Foundation", "Science Fiction");
    printf("Book #1: %s %s\n", pBook->title, pBook->category);
    return 0;
}
```

High mem addr

Environment var.
Argc & argv
pBook

DATA (Gl.var)

TEXT (CODE)

Low mem addr



Övning: Ok?

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {char title[20+1]; char category[20+1];} Book;

Book *init(char theTitle[], char theCategory[]) {
    Book theBook;
    strcpy(theBook.title, theTitle);
    strcpy(theBook.category, theCategory);
    return &theBook;
}

int main(int argc, char **argv)
{
    Book *pBook=init("Foundation", "Science Fiction");
    printf("Book #1: %s %s\n", pBook->title, pBook->category);
    return 0;
}
```

High mem addr

Book #1: p?0??

Environment var.
Argc & argv
pBook

theTitle="Found...
theCategory="Sci...
theBook={"...", "..."}
[Empty memory block]

DATA (Gl.var)

TEXT (CODE)

Low mem addr



Övning: Ok?

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {char title[20+1]; char category[20+1];} Book;

Book *init(char theTitle[], char theCategory[]) {
    Book theBook;
    strcpy(theBook.title, theTitle);
    strcpy(theBook.category, theCategory);
    return &theBook;
}

int main(int argc, char **argv)
{
    Book *pBook=init("Foundation", "Science Fiction");
    printf("Book #1: %s %s\n", pBook->title, pBook->category);
    return 0;
}
```

High mem addr

Environment var.
Argc & argv
pBook

theTitle="Found...
theCategory="Sci...
theBook={"...", "..."} ←

NIX!

DATA (Gl.var)

TEXT (CODE)

Low mem addr

Book #1: @ ??



void *malloc(size_t size)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {char title[20+1]; char category[20+1];} Book;

Book *init(char theTitle[], char theCategory[]) {
    Book *pBook=(Book *)malloc(sizeof(Book));
    strcpy(pBook->title, theTitle);
    strcpy(pBook->category, theCategory);
    return pBook;
}

int main(int argc, char **argv)
{
    Book *pBook=init("Foundation","Science Fiction");
    printf("Book #1: %s %s\n", pBook->title, pBook->category);
    return 0;
}
```

Samma!

High mem addr

Environment var.
Argc & argv
pBook

DATA (Gl.var)

TEXT (CODE)

Low mem addr



void *malloc(size_t size)

NYTT!

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {char title[20+1]; char category[20+1];} Book;

Book *init(char theTitle[], char theCategory[]) {
    Book *pBook=(Book *)malloc(sizeof(Book));
    strcpy(pBook->title, theTitle);
    strcpy(pBook->category, theCategory);
    return pBook;
}

int main(int argc, char **argv)
{
    Book *pBook=init("Foundation","Science Fiction");
    printf("Book #1: %s %s\n", pBook->title, pBook->category);
    return 0;
}
```

High mem addr

Book #1: p?0??

Environment var.
Argc & argv
pBook

theTitle="Found...
theCategory="Sci...
pBook=

{"...", "..."}
↑

DATA (Gl.var)

TEXT (CODE)

Low mem addr

Reserverar plats på heapen, ny egenadministrerad dataarea!

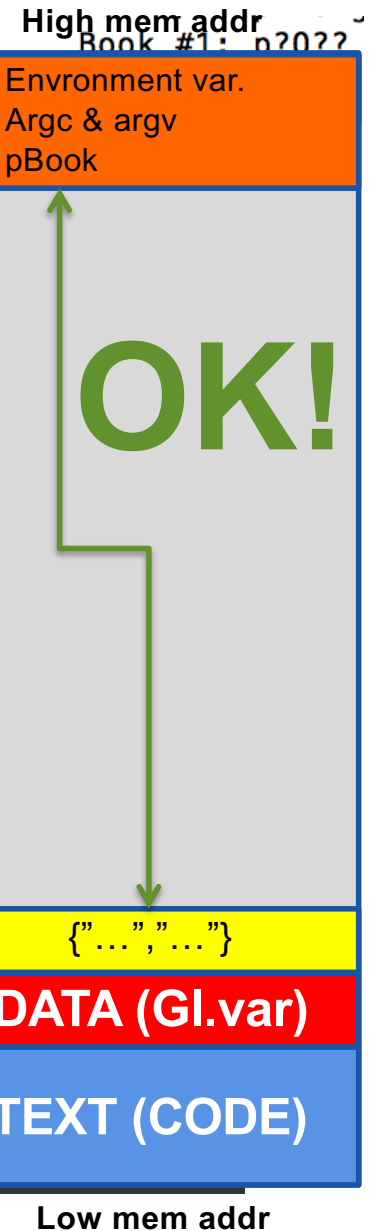


void *malloc(size_t size)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {char title[20+1]; char category[20+1];} Book;

Book *init(char theTitle[], char theCategory[]) {
    Book *pBook=(Book *)malloc(sizeof(Book));
    strcpy(pBook->title, theTitle);
    strcpy(pBook->category, theCategory);
    return pBook;
}

int main(int argc, char **argv)
{
    Book *pBook=init("Foundation","Science Fiction");
    printf("Book #1: %s %s\n", pBook->title, pBook->category);
    return 0;
}
```

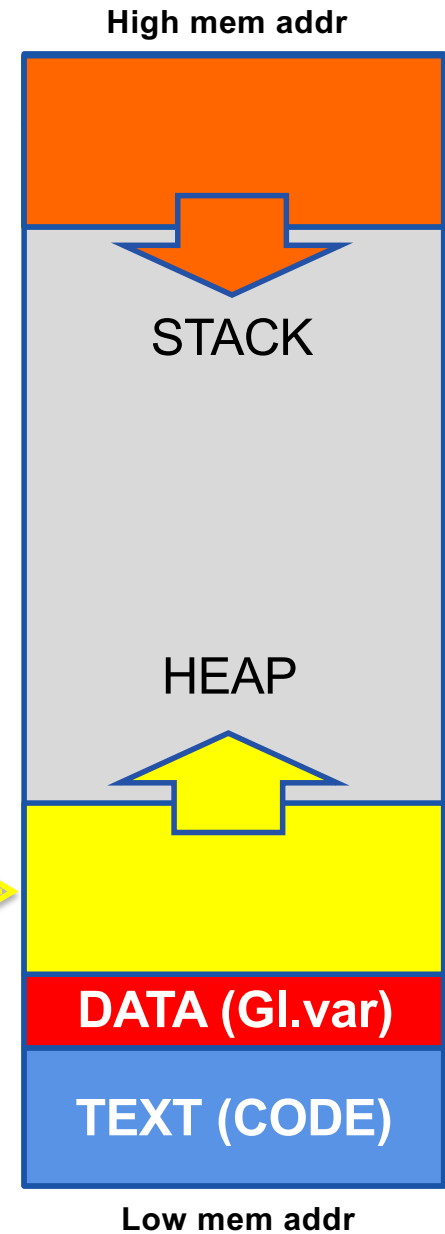


Book #1: Foundation Science Fiction



Begrepp

`malloc(bytes)`



`free();`

Dynamiskt
Allokerat
Minne



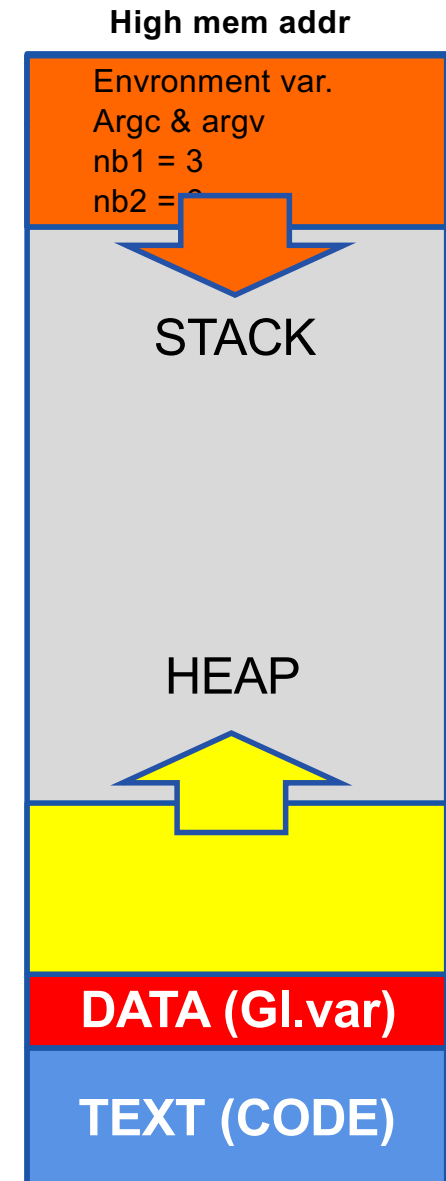
Begrepp

Då finns i minnet:

- Programkoden,
- Globala variabler,
- Stack & Heap

Nu kan processorn börja exekvera instruktionerna i programkoden, vilket innebär att processorns register också blir en del av helheten.

Definition: Vi kallar innehållet i en exekverbar fil för ett **program**. När programmet är laddat i datorns minne och exekveras, eller är redo att exekveras, kallar vi programmet för en **process**!

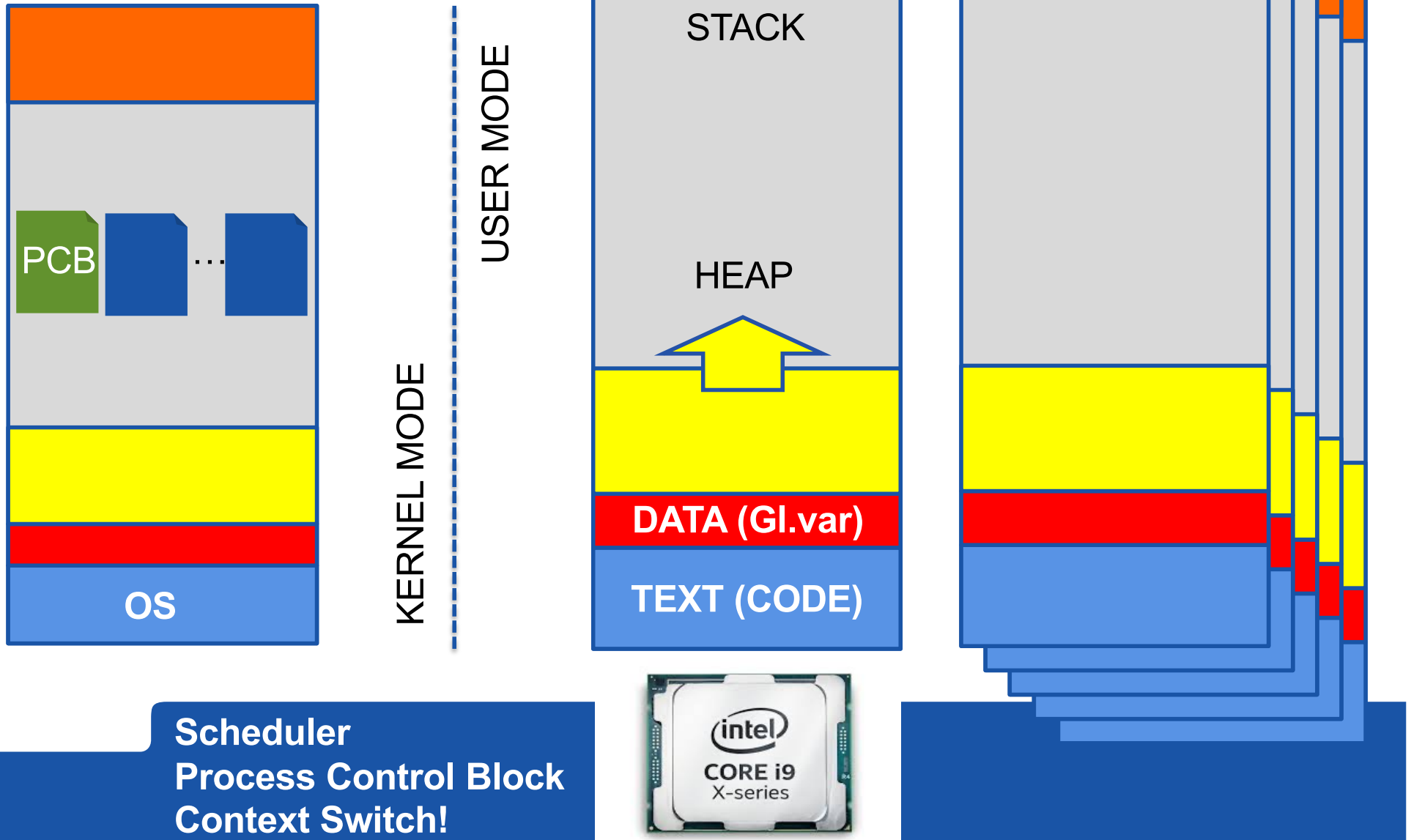


"Allt" som "tillhör" en process kallas för processens machine state!





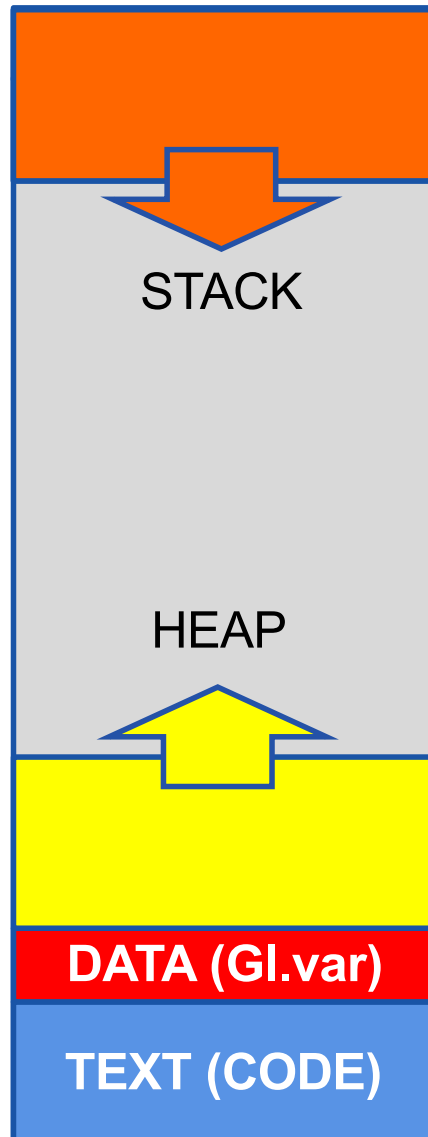
Begrepp



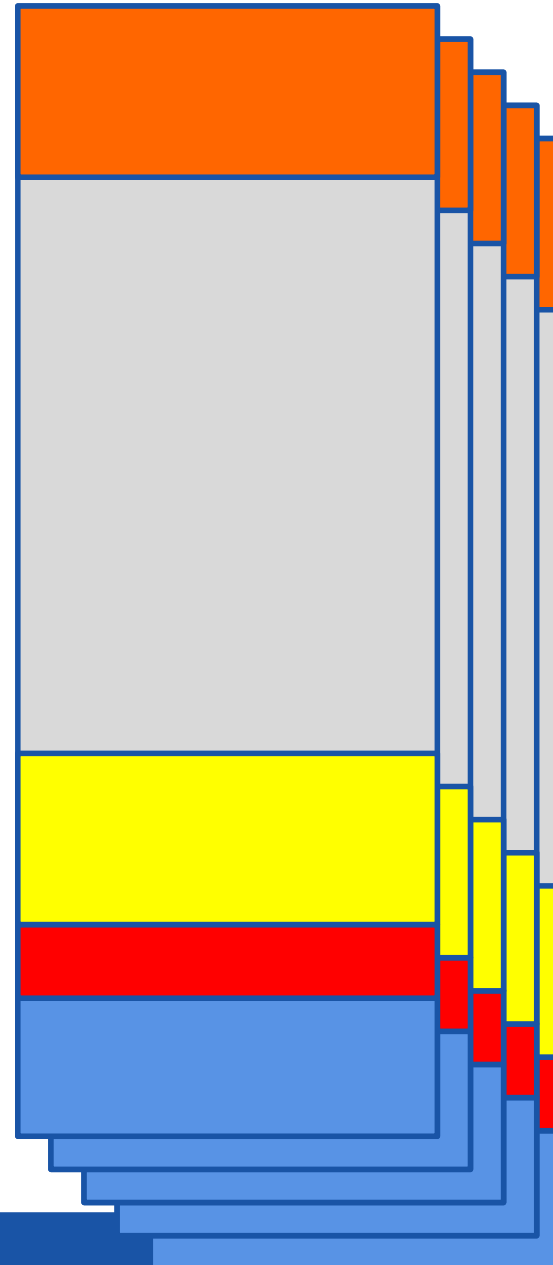


Inbyggda system

RUNNING



READY



Lazily

Eagerly

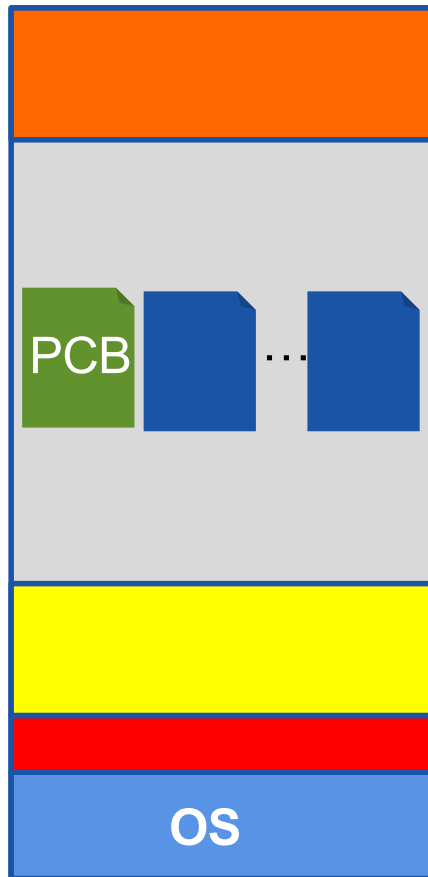


RAM/Hårddisk





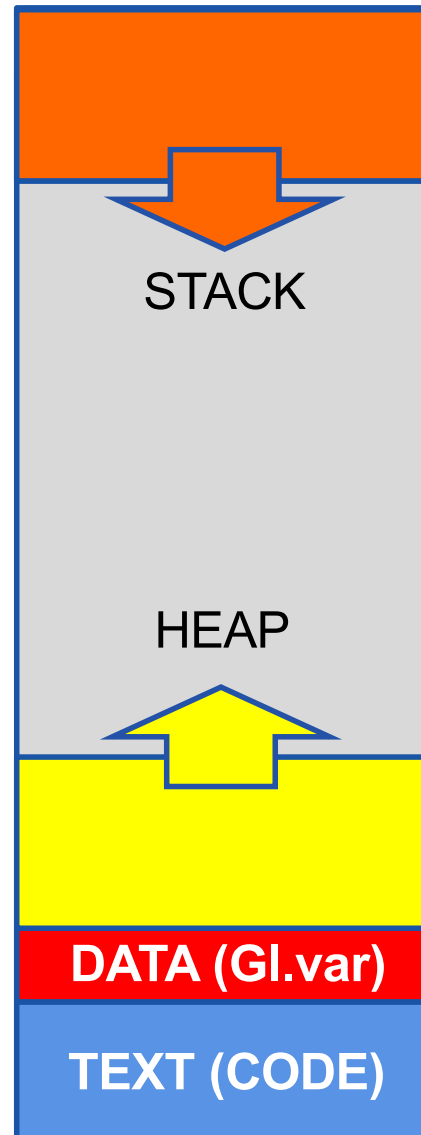
2 Krux!



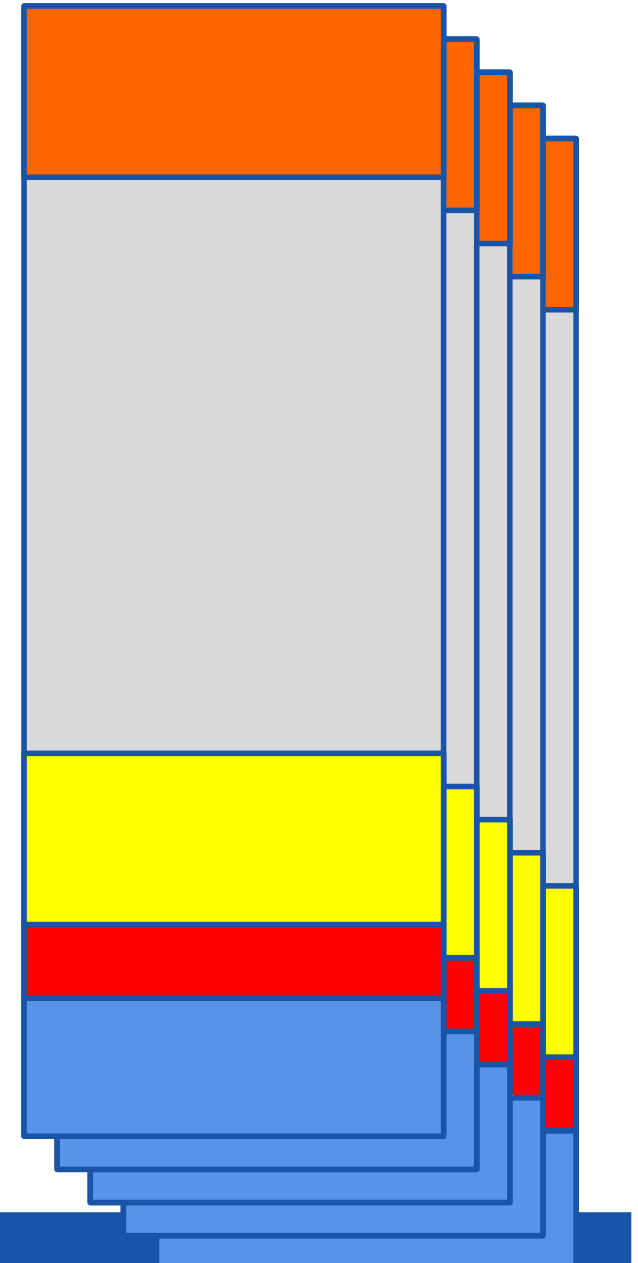
Security
Control

Load...go!

RUNNING



READY



(1:2)

Interpreting vs

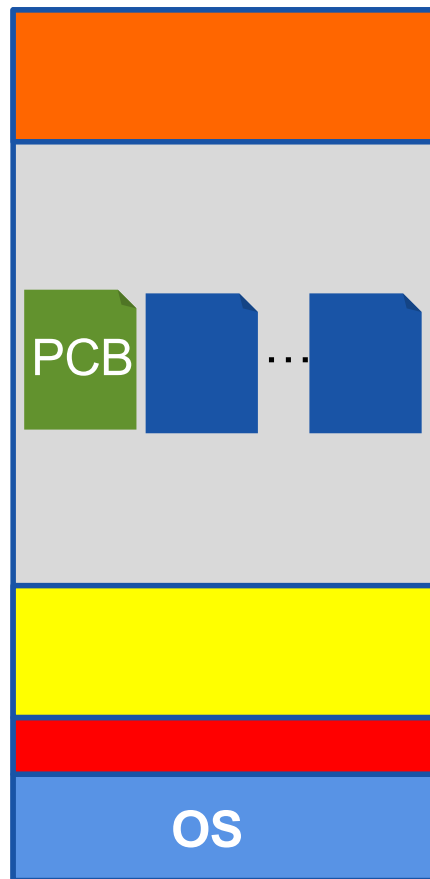
Limited Direct



Execution (LDE)



2 Krux

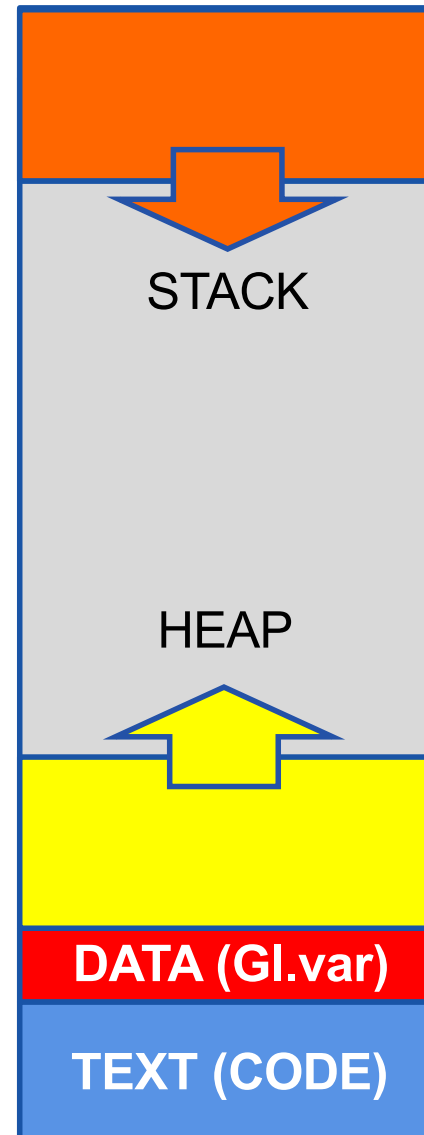


KERNEL MODE

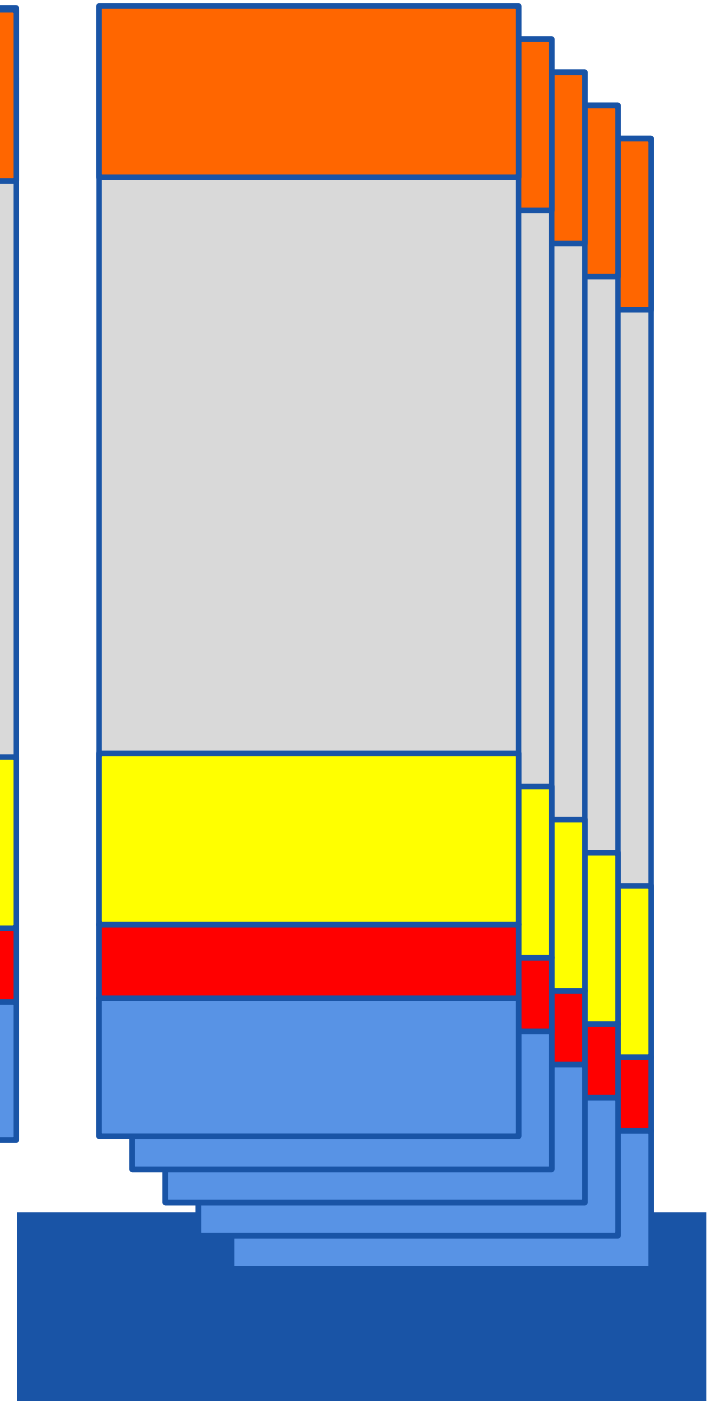
USER MODE

Load...go!

RUNNING



READY



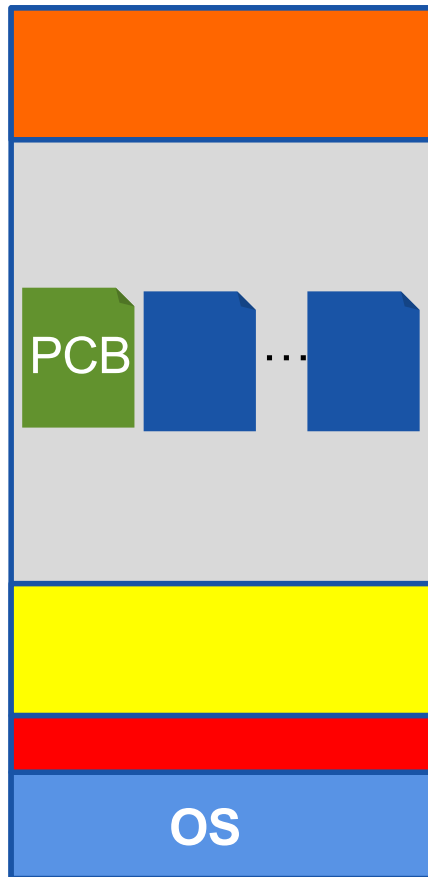
(1:2)

Modes with different
privilege!





Cooperative



KERNEL MODE

USER MODE

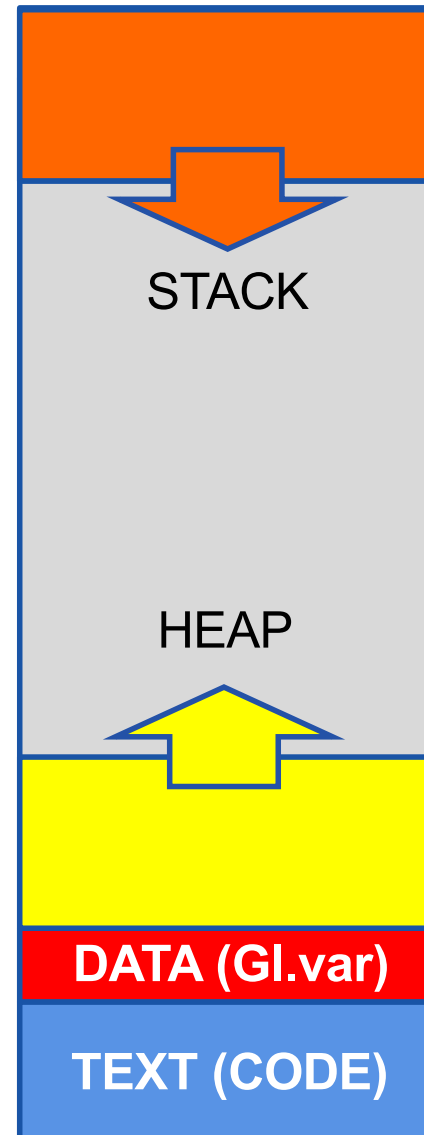
Sys. Call

(2:2)

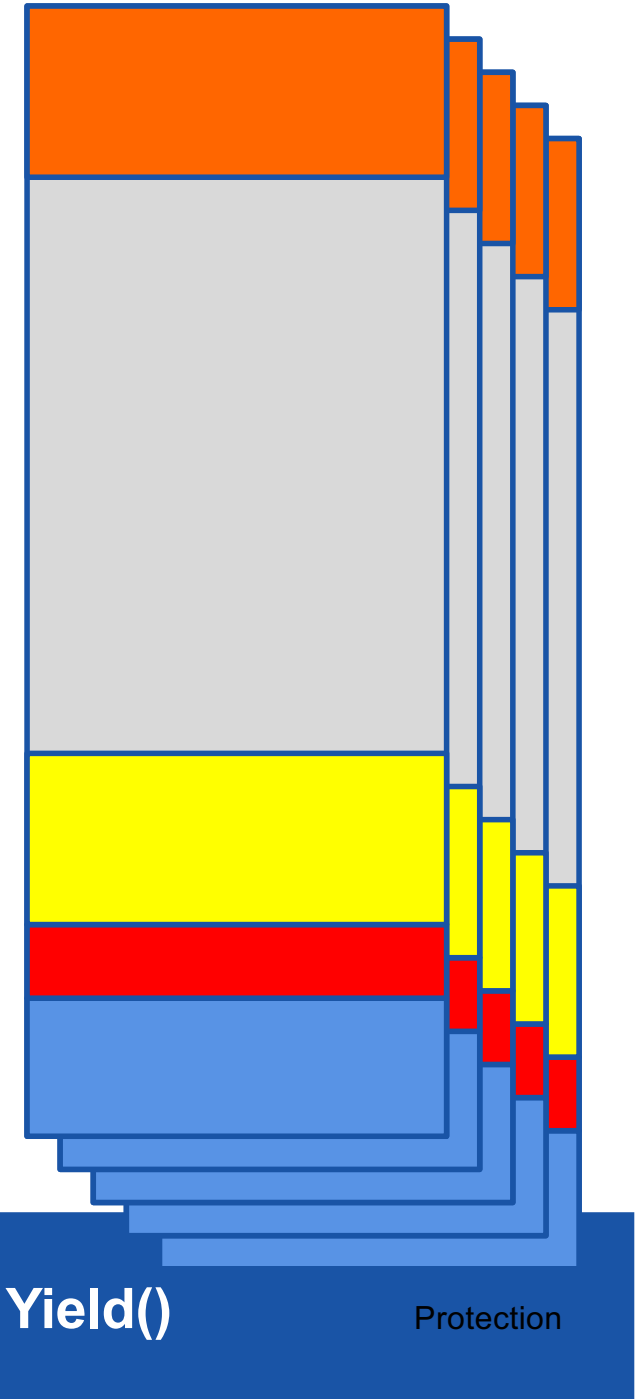
Wait for Sys.Call!



RUNNING



READY

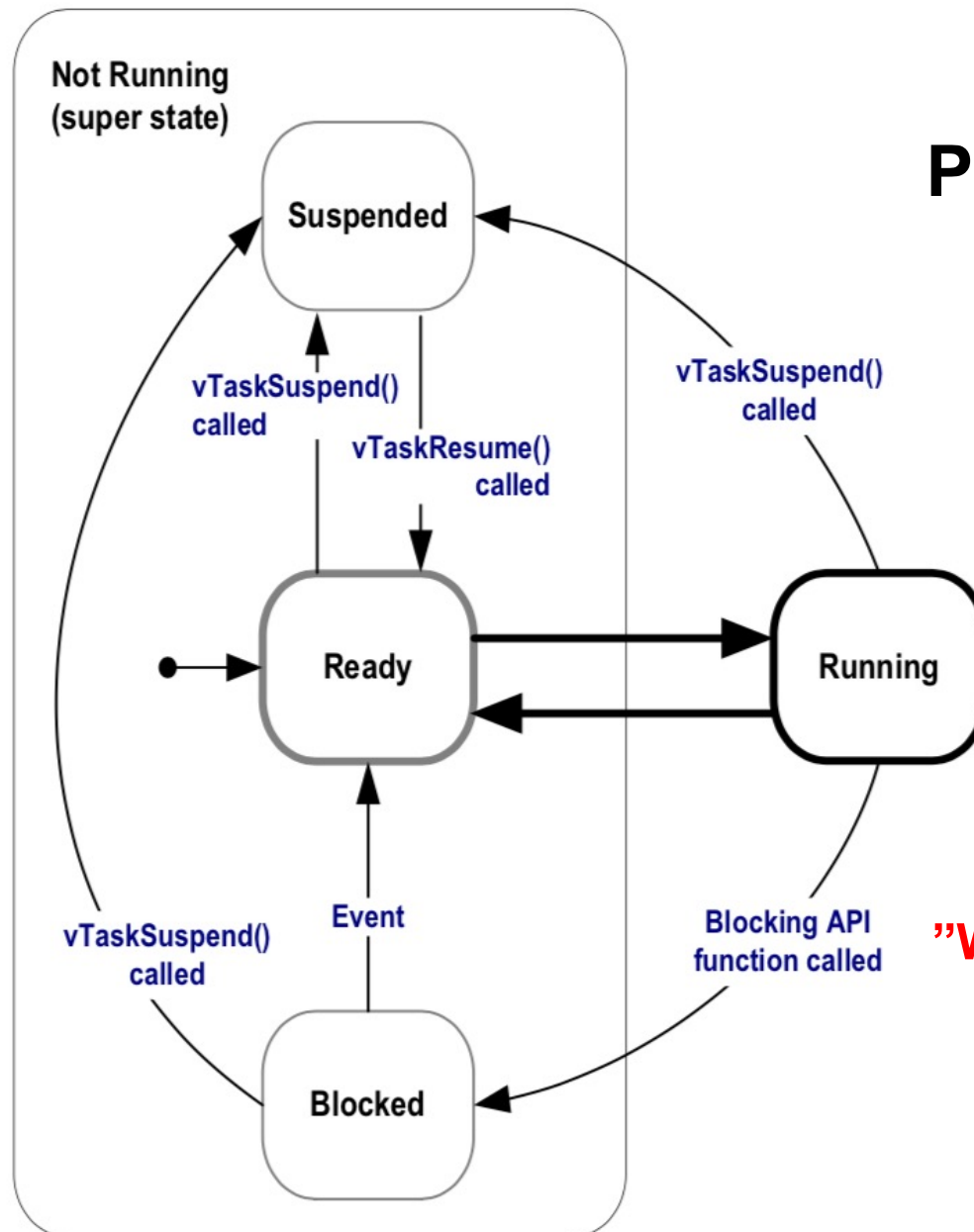




Timer interrupt!



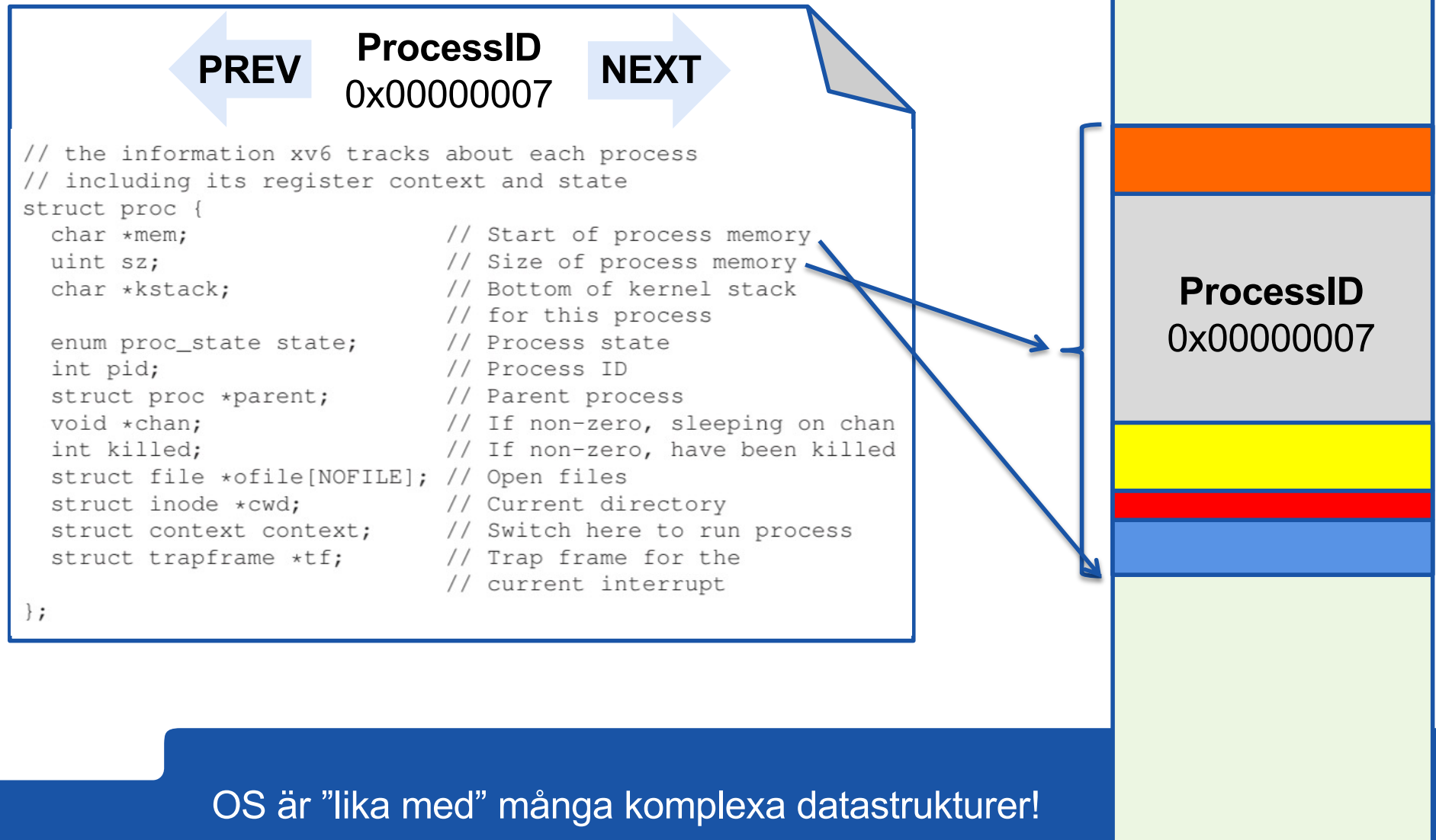
Process states



Scheduling...

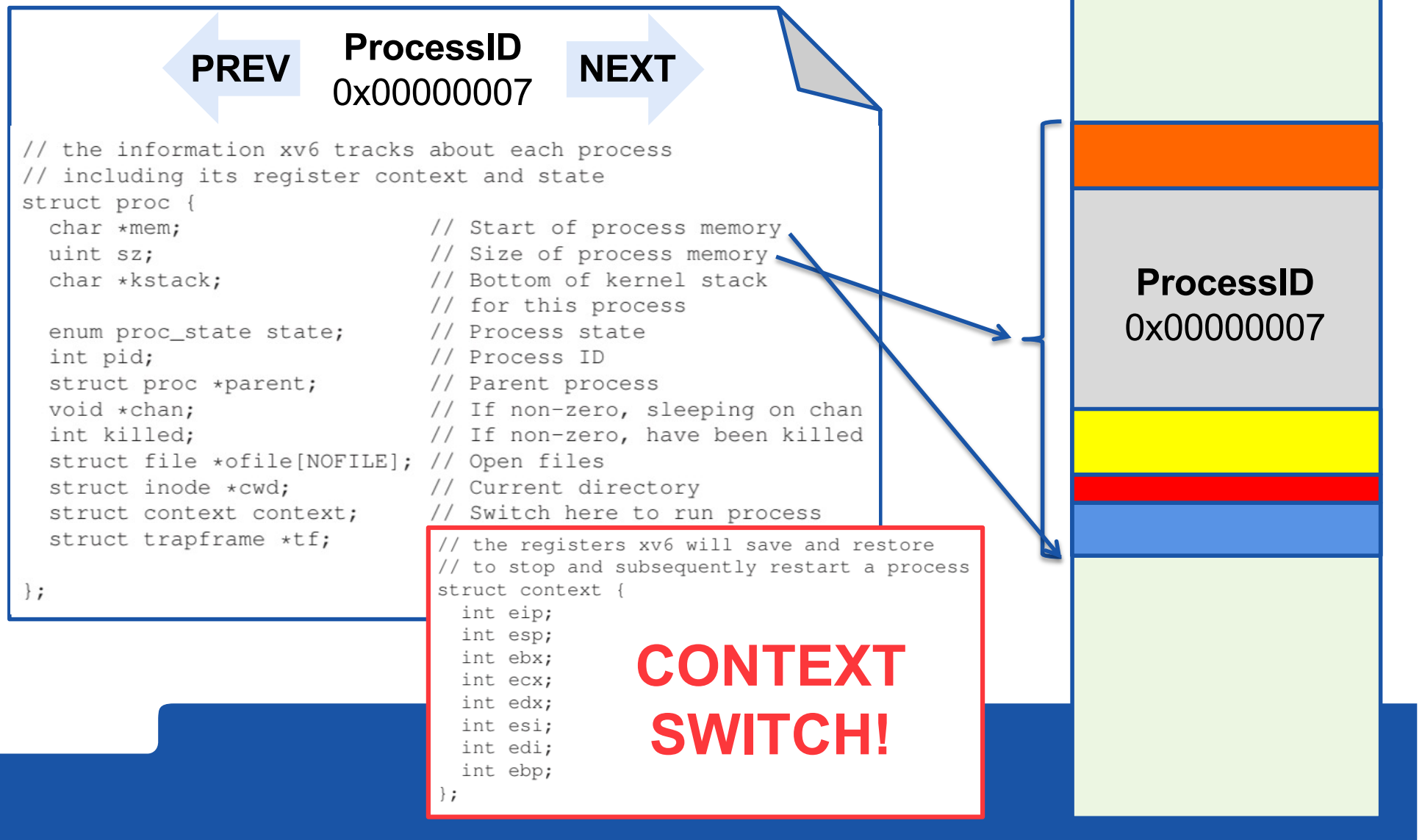


PCB (aka Task Control Block)





PCB (aka Task Control Block)

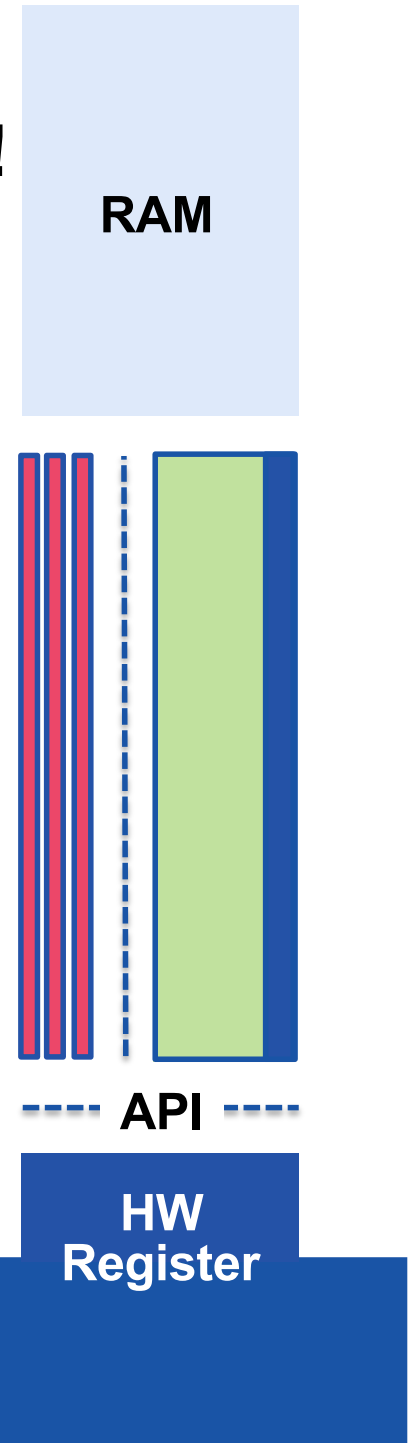




Borde det inte finnas ind. exe. oxo?!

När processorn exekverar applikationskod...

...så består den av maskininstruktioner som processorn direkt kan utföra, även om några enstaka instruktioner är blockerade i user mode. Limited Direct Execution!



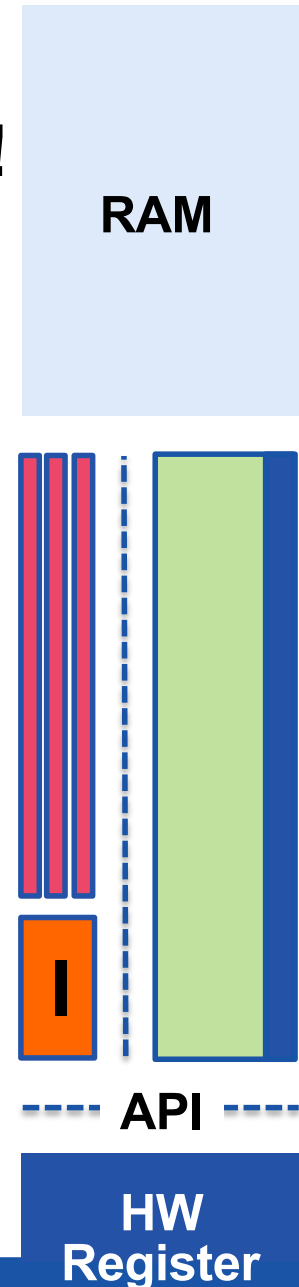


Borde det inte finnas ind. exe. oxo?!

När processorn exekverar applikationskod...

...så består den av maskininstruktioner som processorn direkt kan utföra, även om några enstaka instruktioner är blockerade i user mode. Limitid Direct Execution!

Men applikationskoden kan också bestå av någon form av generell "maskinkod" som en interpreter (virtual machine) tolkar och utför när processorn är tillgänglig!

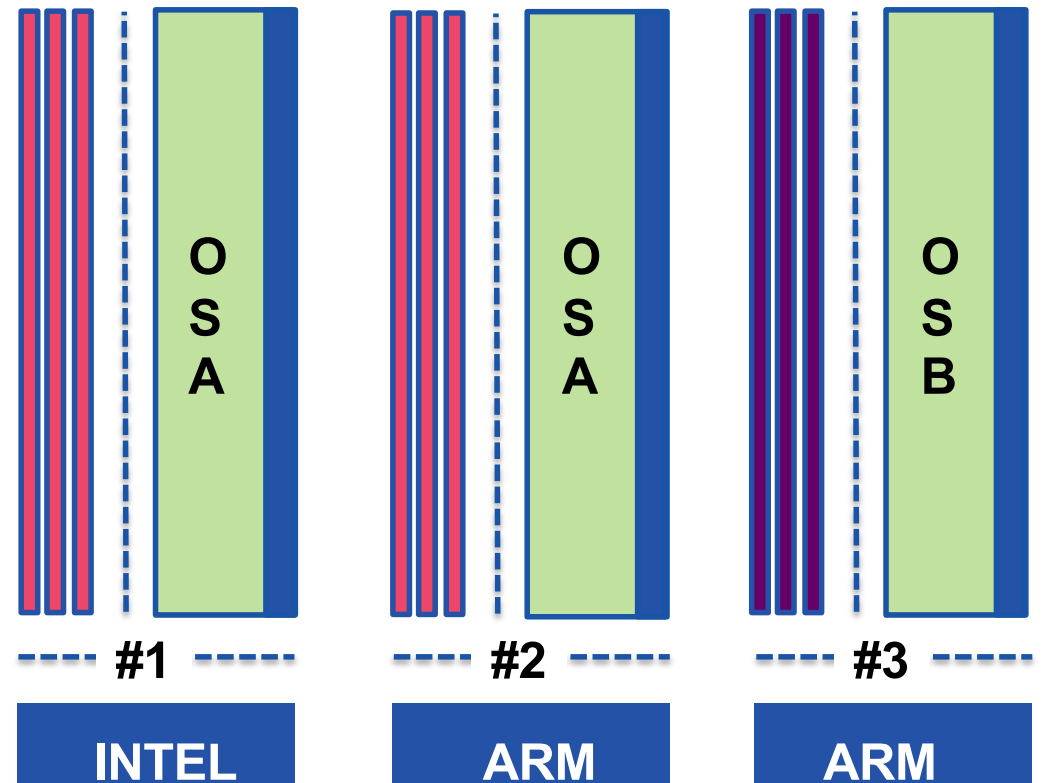


Program skrivna i Java översätts till något som kallas byte code...
...som "körs" (tolkas och utförs) i en virtuell maskin.



Abstraktion...

Typ B program,
men på typ A OS?!



Går inte direkt, även om processorn är av samma typ...
...då OS API:et är olika för alla större operativsystem!



Abstraktion...



Virtual OS B, running on OS A!

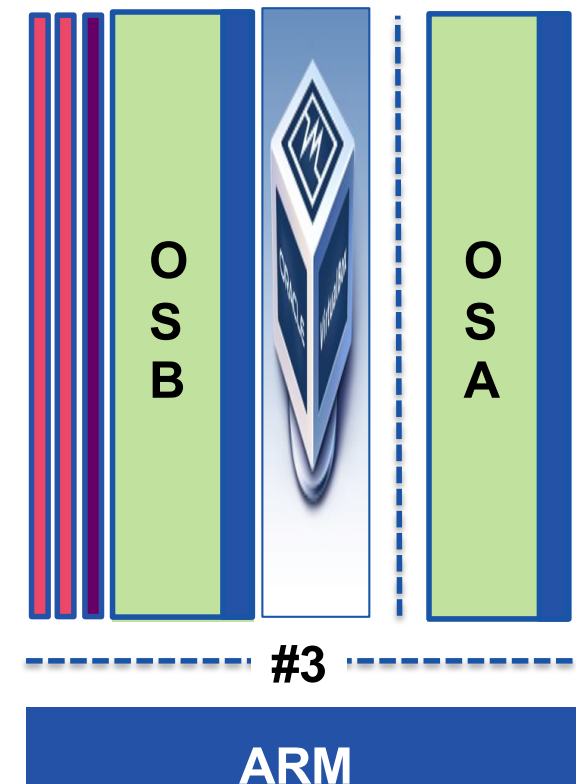
Fortfarande Limited Direct Execution...

...Systemanrop "fångas" av virtualiseringsprogramvaran och översätts till korrekta anrop för grund (host) operativsystemet.

Uppenbart kostar systemanrop mer...

...samt att flera "hela" operativsystem finns samtidigt i minnet...

...OS B kör i user-mod!



Benämns ofta Typ 2 Hypervisor och kan extrapoleras till...



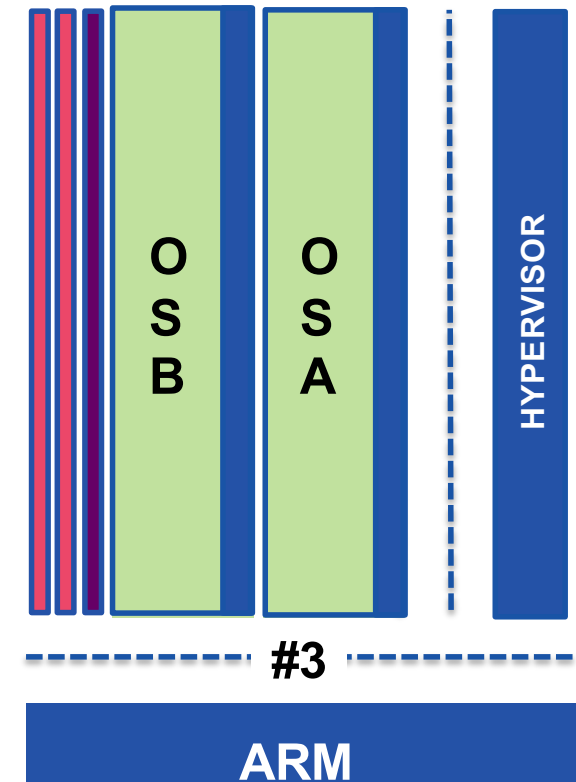
Abstraktion...

VMware
vSphere™ 5.0

Den senaste generations processorer innehåller hårdvarustöd för att dirigera systemanrop till "rätt" OS...

...en mycket begränsad virtualiserings-programvara benämnd Typ 1 Hyper-visor är då allt som behövs!

Både applikationskod och os-kod måste dock vara avsedd för hårdvarans processorn!



För att utnyttja en processor maximalt...
...men med isolation använd hypervisorn också på följande sätt...



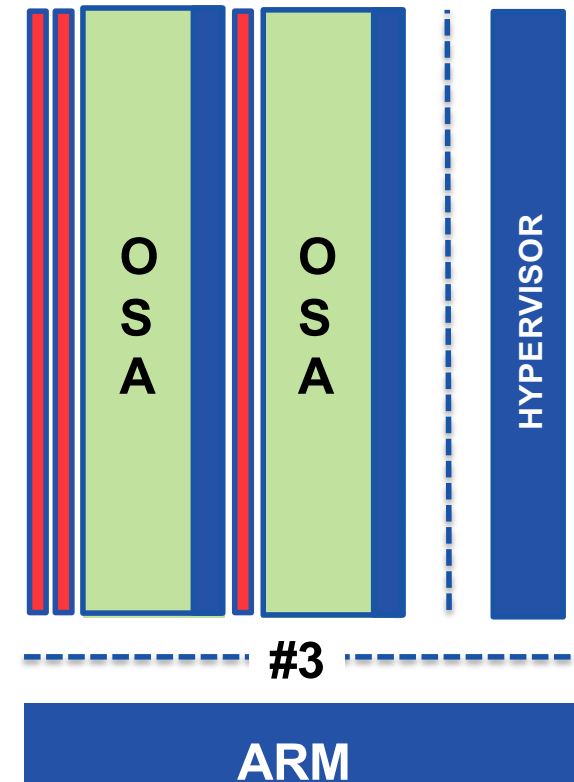
Abstraktion...

VMware
vSphere™ 5.0

T.ex så finns det en komplett installation av ett operativsystem och en databas...

...men också en komplett installation av ett till operativsystem och en web-server applikation...

...bägge helt isolerade från varandra!



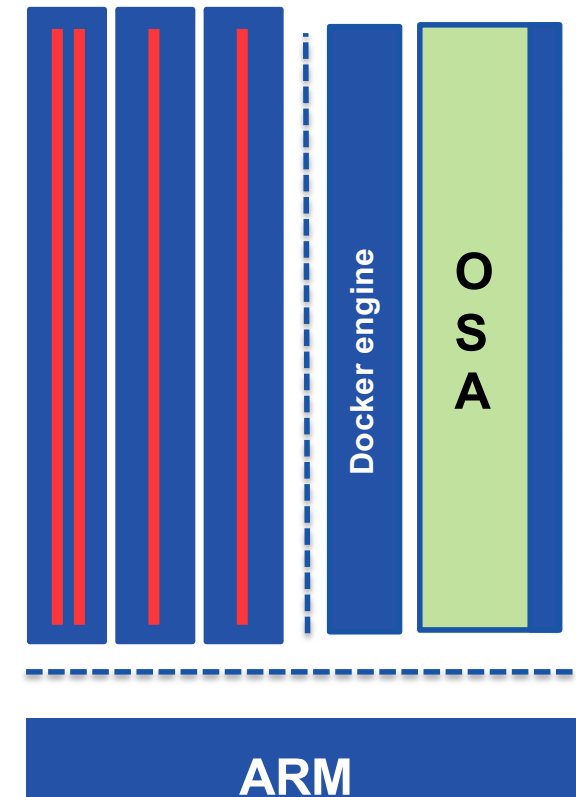
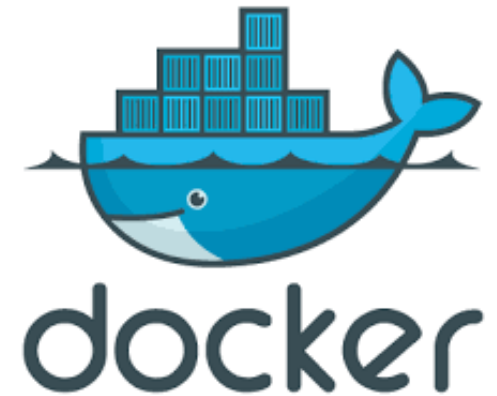
Att ha ett komplett operativsystem för varje applikation kostar en hel del resurser och tar uppenbart tid att starta upp när det behövs...



Abstraktion...

Därför har en Linux-tjänst med tiden vuxit till en möjlighet att ha ett operativsystem och en "motor" som skapar nästan helt isolerade "sand-lådor", benämnda containers, där applikationer kan exekvera åtskilda men dela på operativsystemet!

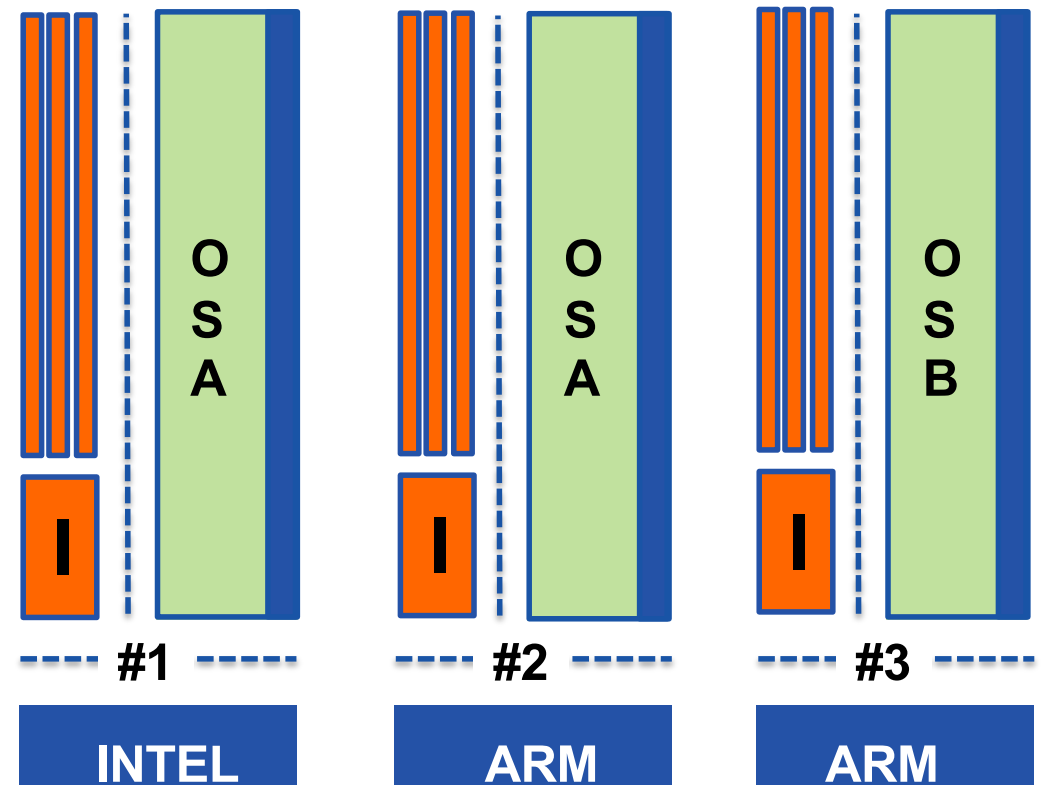
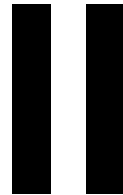
Varje container har ett eget namespace samt allt som krävs i form av bibliotek och inställningar för att applikationen ska fungera även på en annan host!



En för stunden mycket uppmärksammas teknik, tar lite resurser och snabb start, fungerar bra för team som arbetar enligt DevOps.

Abstraktion...

Typ B program,
men på typ A OS?!

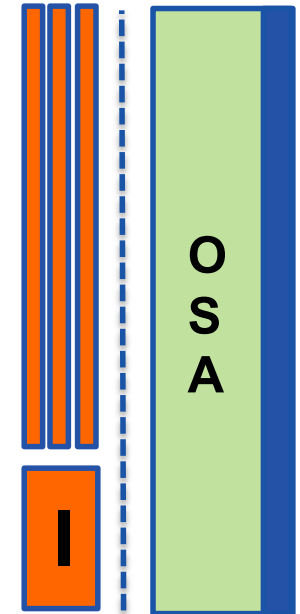
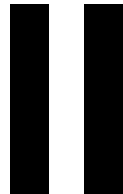


En annan väg att gå är givetvis att applikationskoden är interpreterbar...
...finns en VM för rätt os/processor så fungerar koden (nästan) alltid!



Abstraktion...

Typ B program,
men på typ A OS?!



----- #1 -----



----- #3 -----



En sista utväg är att installera en mjukvara som ger
sken av att hårdvaran är något annat än vad den är!



Fördjupning: Trap

OS @ boot
(kernel mode)

initialize trap table

Hardware

remember address of...
syscall handler

OS @ run
(kernel mode)

Create entry for process list
Allocate memory for program
Load program into memory
Setup user stack with argv
Fill kernel stack with reg/PC
return-from-trap

Hardware

restore regs from kernel stack
move to user mode
jump to main

Program
(user mode)

Run main()
...
Call system call
trap into OS

save regs to kernel stack
move to kernel mode
jump to trap handler

Handle trap
Do work of syscall
return-from-trap

restore regs from kernel stack
move to user mode
jump to PC after trap

...
return from main
trap (via `exit()`)

Free memory of process
Remove from process list

Interrupt
Descriptor
Table
256 entries
0..x1F OS
x80 trap

eax=SC#
syscall/
int 0x80

sysret



2 Stackar per process!

In a Linux system, every user process has 2 stacks, a user stack and a dedicated kernel stack for the process. The user stack resides in the user address space (first 3GB in 32-bit x86 arch.) and the kernel stack resides in the kernel address space (3GB-4GB in 32bit-x86) of the process.

When a user process needs to execute some privileged instruction (a system call) it traps to kernel mode and the kernel executes it on behalf of the user process.

This execution takes place on the process' kernel stack.

The **TSS (Task State Segment)** [↗](#) is used to store the segment selector and stack pointer of the process' kernel stack.

Upon a system call, the user process pushes all caller save registers in the process' user stack and executes **int \$0x80** [↗](#) (0x80 is for system call) instruction. Then the **hardware** (not software) finds the process' kernel stack address from the TSS, loads those values to **%ss** (stack segment selector) and pushes the old stack stack-pointer (**%esp**), old program-counter (**%eip**), old stack segment (**%ss**), code segment(**%cs**), and EFLAGS registers in the **process' kernel stack**. Hence, any operation requiring stack access uses this stack (not the user stack). After execution is over, the hardware pops the saved values from the kernel stack to their respective registers and resumes in the user stack.

For a trap execution, the user stack can't be used because it might be corrupted by malicious user process. The kernel stack is used by kernel code only, so it is safe.

(Också bra för systemanrop som tar tid...)



2 Stackar per process!

Många systemanrop tar tid att utföra...

...i stället för att vänta schemalägger OS:et helt enkelt en annan process som får köra under tiden...

...det innebär att ett systemanrop inte bara sparar undan de viktigaste registren på kärnstacken utan att ett systemanrop ofta medför att en komplett context switch utförs!

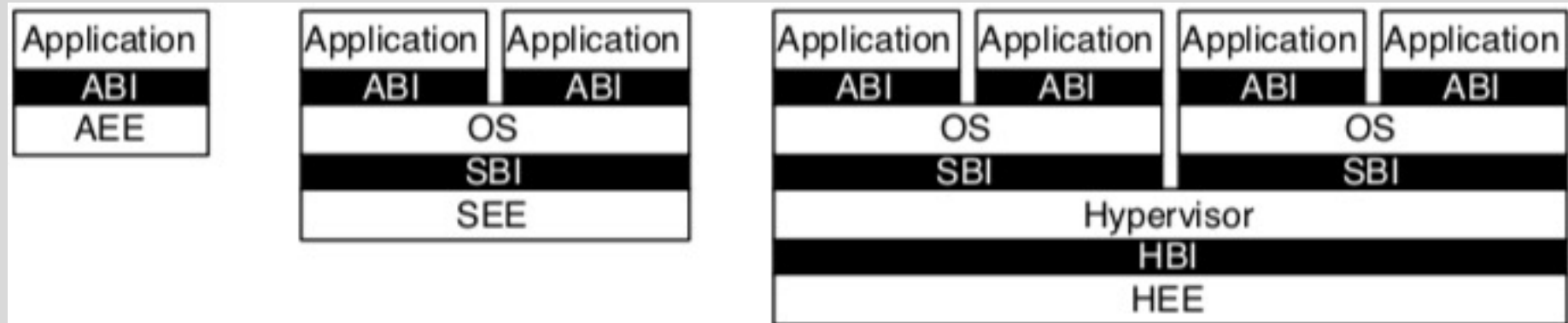
...processen som utförde anropet blir "suspended/blocked" till dess att OS:et får information om att önskad tjänst är genomförd, som leder till att ursprungsprocessen märks Ready och tävlar om att få köra vid nästa context switch.



RISC-V

Level	Encoding	Name	Abbreviation
0	00	User/ Application	U
1	01	Supervisor	S
2	10	Hypervisor	H
3	11	Machine	M

Number of levels	Supported Modes	Intended Usage
1	M	Simple embedded systems
2	M, U	Secure embedded systems
3	M, S, U	Systems running Unix-like operating systems
4	M, H, S, U	Systems running Type-1 hypervisors



xBI=xBinary Interface, xEE=xExecutorial Environment



RISC-V

Number	Privilege	Name	Description
User Trap Setup			
0x000	URW	ustatus	User status register. ←.....
0x004	URW	uie	User interrupt-enable register.

Number	Privilege	Name	Description
Supervisor Trap Setup			
0x100	SRW	sstatus	Supervisor status register. ←.....

Number	Privilege	Name	Description
Hypervisor Trap Setup			
0x200	HRW	hstatus	Hypervisor status register. ←.....

Number	Privilege	Name	Description
Machine Information Registers			
0x202	MRO	mvendorid	Vendor ID.
0x203	MRO	marchid	Architecture ID.
0x204	MRO	mimpid	Implementation ID.
0x205	MRO	mhartid	Hardware thread ID.

Number	Privilege	Name	Description
Machine Trap Setup			
0x300	MRW	mstatus	Machine status register.
0x301	MRW	misa	ISA and extensions
0x302	MRW	medeleg	Machine exception delegation register.
0x303	MRW	mideleg	Machine interrupt delegation register.
0x304	MRW	mie	Machine interrupt-enable register.
0x305	MRW	mtvec	Machine trap-handler base address.

Number	Privilege	Name	Description
Machine Trap Handling			
0x340	MRW	mscratch	Scratch register for machine trap handling.
0x341	MRW	mepc	Machine exception program counter.
0x342	MRW	mcause	Machine trap cause.
0x343	MRW	mbadaddr	Machine bad address.
0x344	MRW	mip	Machine interrupt pending.

Number	Privilege	Name	Description
Machine Protection and Translation			
0x380	MRW	mbase	Base register.
0x381	MRW	mbound	Bound register.
0x382	MRW	mibase	Instruction base register.
0x383	MRW	mibound	Instruction bound register.
0x384	MRW	mdbase	Data base register.
0x385	MRW	mdbound	Data bound register.

CSR Address			Hex	Use and Accessibility
[11:10]	[9:8]	[7:6]	2 * 2 * 256	
User CSRs				
00	00	XX	0x000-0x0FF	Standard read/write
01	00	XX	0x400-0x4FF	Standard read/write
10	00	XX	0x800-0x8FF	Non-standard read/write
11	00	00-10	0xC00-0xCBF	Standard read-only
11	00	11	0xCC0-0xCFF	Non-standard read-only
Supervisor CSRs				
00	01	XX	0x100-0x1FF	Standard read/write
01	01	00-10	0x500-0x5BF	Standard read/write
01	01	11	0x5C0-0x5FF	Non-standard read/write
10	01	00-10	0x900-0x9BF	Standard read/write shadows
10	01	11	0x9C0-0x9FF	Non-standard read/write shadows
11	01	00-10	0xD00-0xDBF	Standard read-only
11	01	11	0xDC0-0xDFF	Non-standard read-only
Hypervisor CSRs				
00	10	XX	0x200-0x2FF	Standard read/write
01	10	00-10	0x600-0x6BF	Standard read/write
01	10	11	0x6C0-0x6FF	Non-standard read/write
10	10	00-10	0xA00-0xABF	Standard read/write shadows
10	10	11	0xAC0-0xAFF	Non-standard read/write shadows
11	10	00-10	0xE00-0xEBF	Standard read-only
11	10	11	0xEC0-0xEFF	Non-standard read-only
Machine CSRs				
00	11	XX	0x300-0x3FF	Standard read/write
01	11	00-10	0x700-0x79F	Standard read/write
01	11	10	0x7A0-0x7AF	Standard read/write debug CSRs
01	11	10	0x7B0-0x7BF	Debug-mode-only CSRs
01	11	11	0x7C0-0x7FF	Non-standard read/write
10	11	00-10	0xB00-0xBBF	Standard read/write shadows
10	11	11	0xBC0-0xBFF	Non-standard read/write shadows
11	11	00-10	0xF00-0xFBF	Standard read-only
11	11	11	0xFC0-0xFFF	Non-standard read-only



RISC-V

POR



BL



```
165 _start:
166
167     csrr CSR_MSTATUS, MSTATUS_MIE
168     /* Jump to logical address first to ensure correct operation of RAM region */
169     la      a0, _start
170     li      a1, 1
171     slli    a1, a1, 29
172     bleu    a1, a0, _start0800
173     srli    a1, a1, 2
174     bleu    a1, a0, _start0800
175     la      a0, _start0800
176     add     a0, a0, a1
177     jr      a0
178
179 _start0800:
180
181     /* Set the the NMI base to share with mtvec by setting CSR_MMISC_CTL */
182     li t0, 0x200
183     csrrs CSR_MMISC_CTL, t0
184
185     /* Intial the mtvt*/
186     la t0, vector_base
187     csrw CSR_MTVT, t0
188
189     /* Intial the mtvt2 and enable it*/
190     la t0, irq_entry
191     csrw CSR_MTVT2, t0
192     csrrs CSR_MTVT2, 0x1
193
194     /* Intial the CSR MTVEC for the Trap and NMI base addr*/
195     la t0, trap_entry
196     csrw CSR_MTVEC, t0
197
198 #ifdef __riscv_flen
199     /* Enable FPU */
200     li t0, MSTATUS_FS
201     csrrs mstatus, t0
202     csrrw fcsr, x0
203 #endif
204
205 .option push
206 .option norelax
207     la gp, __global_pointer$
208 .option pop
209     la sp, _sp
```



Väldigt
Många
Rader



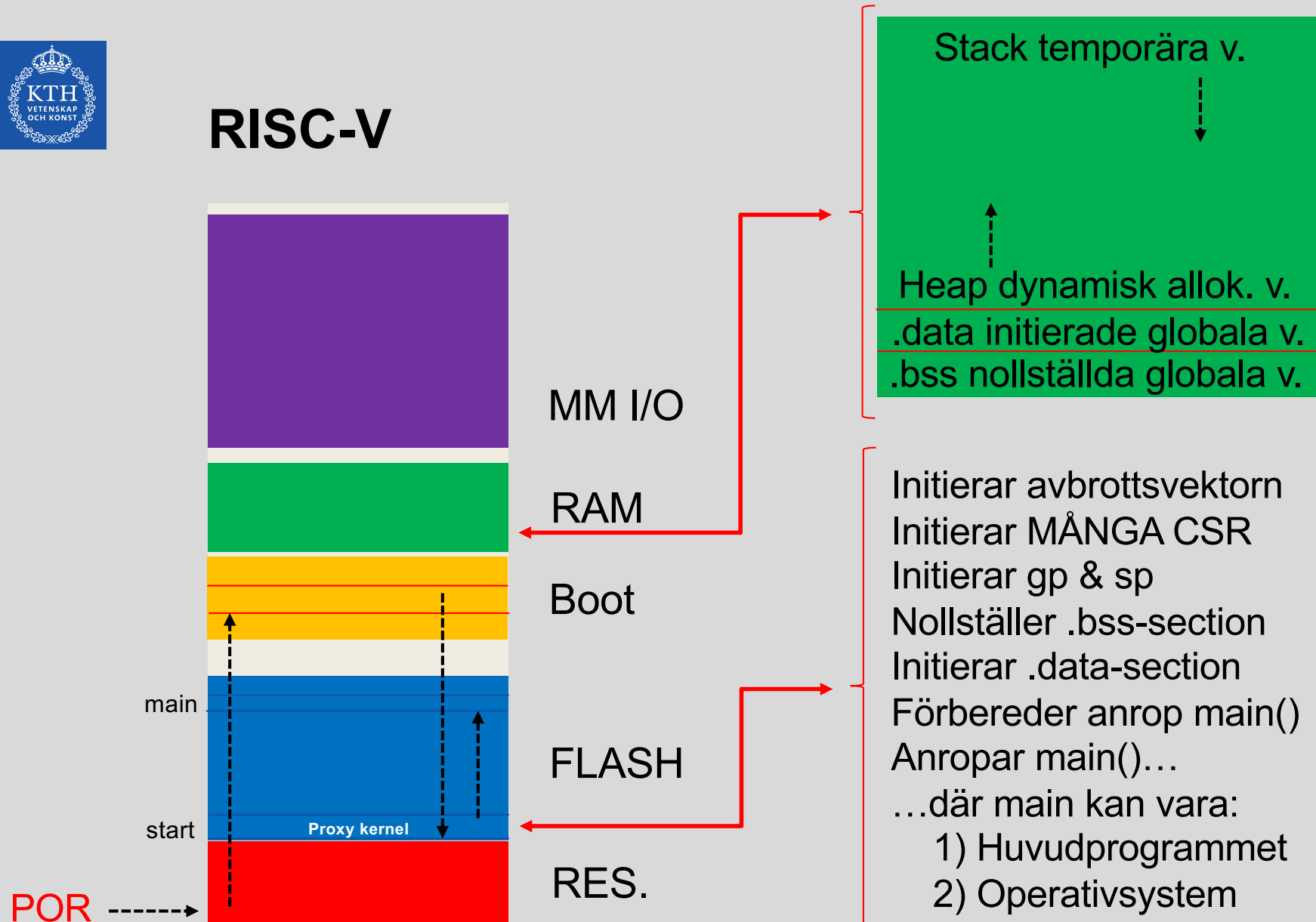
```
72 vector_base:
73     j _start
74     .align    2
75     .word     0
76     .word     0
77     .word     eclic_msip_handler
78     .word     0
79     .word     0
80     .word     0
81     .word     eclic_mt看ip_handler
82     .word     0
83     .word     0
84     .word     0
85     .word     0
86     .word     0
87     .word     0
88     .word     0
89     .word     0
90     .word     0
91     .word     eclic_bwei_handler
```

```
240     /* argc = argv = 0 */
241     li a0, 0
242     li a1, 0
243     call main
244     tail exit
245
246 1:
247     j 1b
```





RISC-V





Begrepp

Bare metal, Batch Job
Command Line Interface (CLI)
Kernel, Limited Direct Execution
Supervisor/User mode
System Call, Trap
Virtualization
Mechanisms & Policies
Concurrency
Persistence
Time Sharing/Multiprogramming
Text, Data, Stack & Heap
Stack Frame (in depth)
Reentrant
Program, Process & Machine State
Process Control Block (PCB)
Context Switch & Scheduler
Cooperative vs. Preemptive
Process Stat
Ready, Running & Blocked
Virtual Machine, Interpreting
Hypervisor
Boot/Context Switch in depth





History

0.1 AC 1806xx Komponenter

0.5 AC 1807xx Allt utom fördjupning klart

0.6 AC 180807 Slutgiltig form, fördjupning ej klar.

0.9 AC 181123 Fördjupningar slutförda.

1.0 AC 181221 Finputsning.

1.1 AC 210318 RISC-V uppdatering.